

EMPRESAIQ

Agentes Inteligentes Locais com IA

Do Zero ao Agente Inteligente

sem Cloud, sem Custos, sem Limites

© 2026 EmpresaIQ

Error response

Error code: 404

Message: File not found.

Error code explanation: 404 - Nothing matches the given URI.

Introdução

"A inteligência artificial mais poderosa não é a que vive na cloud — é a que trabalha para si, no seu computador, sem pedir licença a ninguém."

Porquê este livro?

Imagine ter um assistente inteligente na sua empresa que responde perguntas, analisa documentos, redige relatórios e executa tarefas automaticamente — **sem enviar um único byte de dados para a internet, sem pagar subscrições mensais e sem depender de nenhum fornecedor externo.**

Isso é exactamente o que vamos construir juntos neste livro.

Nos últimos anos, ferramentas como o ChatGPT e o Google Gemini tornaram a IA acessível a toda a gente. Mas há um problema: **os seus dados saem do seu computador.** Contratos, propostas, dados de clientes, informação confidencial — tudo vai para servidores de outras empresas, noutros países, sujeitos a políticas que mudam sem aviso.

Existe uma alternativa. Com os avanços recentes em modelos de linguagem *open source* e em ferramentas como o **Ollama**, hoje é possível correr um agente de IA genuinamente inteligente num PC

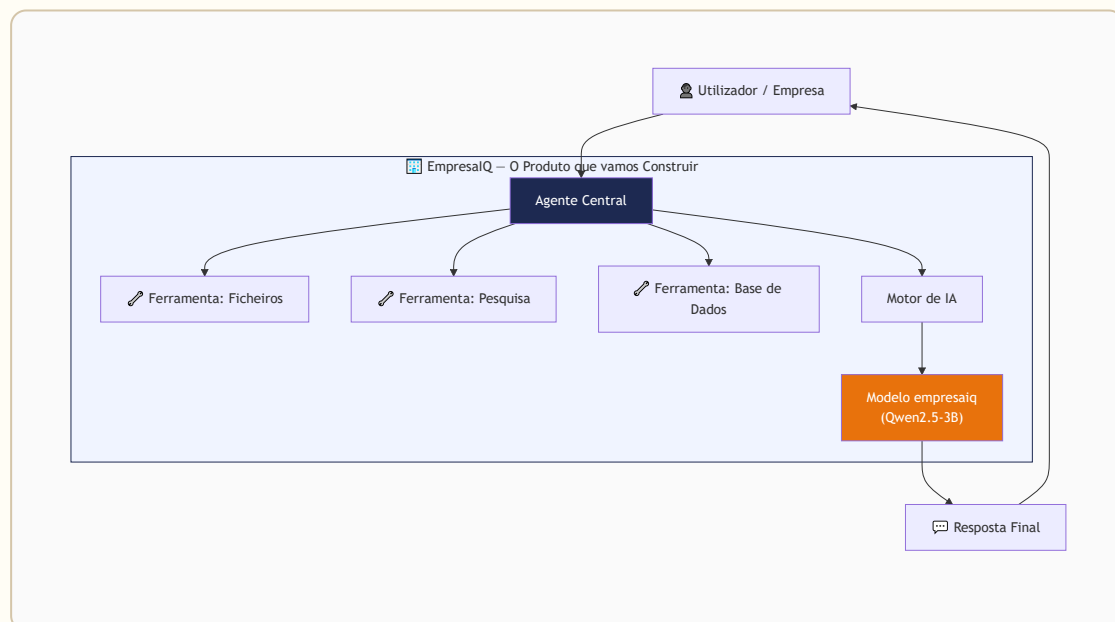
normal com apenas 8 GB de RAM — sem GPU, sem cloud, sem custos recorrentes.

Este livro mostra-lhe como fazer isso, passo a passo, mesmo que nunca tenha trabalhado com IA antes.

O que é o EmpresaIQ?

Ao longo deste livro, vamos construir um produto chamado **EmpresaIQ** — um sistema de agentes de inteligência artificial local, desenhado para o contexto empresarial português.

O EmpresaIQ não é apenas um exemplo académico. É uma solução real, que pode instalar, personalizar e usar na sua empresa. Cada capítulo adiciona uma nova peça ao sistema, até termos um agente completo e funcional.



O sistema final terá:

Componente	O que faz	Quando é construído
Motor de IA (Ollama)	Servidor local que corre o modelo de linguagem	Caps. 8–9
Modelo empresaiq (Qwen2.5-3B)	O "cérebro" do agente, personalizado para o EmpresaIQ	Cap. 9
Ferramentas	Permitem ao agente agir no mundo real	Cap. 10
Agente ReAct	Raciocina e decide que ferramentas usar	Cap. 11
Interface de Chat	Conversa em tempo real com o agente	Cap. 13
Memória Conversacional	Lembra-se do contexto da conversa	Cap. 18
Ontologia de Conhecimento	Estrutura o saber da empresa	Cap. 19

Como este livro está organizado

Este livro está dividido em quatro partes. Pode lê-lo do início ao fim, ou saltar para a parte que mais lhe interessa.

Parte I — Fundamentos (Caps. 1–5)

"O que é tudo isto e porque faz sentido?"

Antes de instalar qualquer coisa, vamos perceber os conceitos. O que é um agente de IA? Porque usar IA local em vez da cloud? Que hardware precisamos? Que modelo escolher? Estes capítulos respondem a todas estas perguntas com linguagem simples e sem jargão desnecessário.

Parte II — Instalação e Configuração (Caps. 6–9)

"Preparar o terreno para o EmpresaIQ"

Vamos instalar o Python, configurar o ambiente de desenvolvimento, instalar o Ollama (o servidor local de IA) e criar o modelo EmpresaIQ personalizado. No final desta parte, o motor do EmpresaIQ estará a funcionar.

Parte III — Construção do Agente (Caps. 10–12)

"Dar vida ao EmpresaIQ"

Aqui construímos o coração do sistema: as ferramentas que o agente usa, o agente em si com capacidade de raciocínio, e as optimizações para correr bem em qualquer CPU.

Parte IV — Produção e Evolução (Caps. 13–20)

"Tornar o EmpresaIQ robusto e inteligente"

Interface de chat, automatização por agendamento, segurança e privacidade, melhorias futuras, suporte ao modelo Qwen2.5, memória conversacional, ontologias de conhecimento e, por fim, um olhar sobre o caminho percorrido.

Para quem é este livro

Este livro foi escrito a pensar em pessoas sem experiência prévia em inteligência artificial. Se sabe ligar um computador e já ouviu falar de ChatGPT, tem tudo o que precisa para começar.

Mais especificamente, este livro é para:

- **Empreendedores e gestores** que querem usar IA nas suas empresas sem depender de fornecedores externos
- **Consultores e advogados** que trabalham com informação confidencial e não podem arriscar expô-la
- **Técnicos de TI** que querem perceber e implementar soluções de IA locais
- **Programadores iniciantes** que querem aprender Python e IA ao mesmo tempo, com um projecto real
- **Curiosos** que querem perceber como a IA funciona "por dentro"

:::info Não precisa de ser programador Os exemplos de código são explicados linha a linha. Mesmo que nunca tenha programado, conseguirá seguir este guia. O mais importante é a curiosidade. :::

O que precisa para começar

Para construir o EmpresaIQ, vai precisar de:

Requisito	Mínimo	Recomendado
Sistema Operativo	Windows 10, Ubuntu 20.04, macOS 12	Windows 11 / Ubuntu 22.04
RAM	8 GB	16 GB
Espaço em Disco	6 GB livres	10 GB livres
Python	3.11	3.11 ou 3.12
Internet	Apenas para instalação inicial	—

:::warning GPU não é necessária Ao contrário do que se pensa, não precisa de uma placa gráfica cara. O EmpresaIQ foi desenhado para correr exclusivamente em CPU, tornando-o acessível a praticamente qualquer computador de escritório. :::

Convenções usadas neste livro

Ao longo do livro, vai encontrar vários tipos de caixas informativas:

:::tip Dica Sugestões práticas para melhorar o seu trabalho ou poupar tempo. :::

:::info Nota Informação adicional útil, mas não obrigatória para continuar. :::

:::warning Atenção Algo importante que pode causar erros se ignorado. :::

:::danger Cuidado Situações que podem comprometer a segurança ou os seus dados. :::

O código é sempre apresentado com o nome do ficheiro indicado:

```
exemplo.py
```

```
# Isto é uma linha de código Python  
print("EmpresaIQ está a funcionar!")
```

Os comandos para executar no terminal aparecem assim:

```
python agente_local.py
```

Antes de começar: uma nota sobre IA

Se nunca trabalhou com inteligência artificial, aqui está a analogia mais simples que existe:

Um modelo de linguagem é como um funcionário muito bem lido — leu milhões de livros, artigos e documentos, e consegue responder a quase qualquer pergunta. Mas sozinho, só fala. Um **agente** é esse mesmo funcionário equipado com ferramentas: um telefone, acesso a ficheiros, uma calculadora. Agora já pode *agir*, não apenas falar.

O **EmpresaIQ** é esse agente — inteligente, equipado, e completamente seu.

Vamos começar.

Capítulo seguinte: *2. Porque usar IA Local* →

Porque usar IA Local

"Dar os seus dados confidenciais a um serviço de cloud é como contratar um assistente que envia cópias dos seus documentos para o patrão dele."

O problema que ninguém fala abertamente

Quando usa o ChatGPT, o Claude ou o Google Gemini para analisar um contrato, redigir uma proposta ou responder a uma questão sobre um cliente, **o que acontece com esses dados?**

A resposta honesta: saem do seu computador, viajam pela internet, são processados em servidores de empresas americanas, e ficam sujeitos às políticas de privacidade dessas empresas — políticas que podem mudar a qualquer momento.

Para uso pessoal e criativo, isso pode ser aceitável. Para uma empresa que trabalha com dados sensíveis, **não é**.

Dois mundos completamente diferentes

🏠 EmpresarialQ – IA Local

Os seus Dados

Processamento interno

Modelo no seu PC

Resposta

☁ IA na Cloud – O que acontece hoje

Os seus Dados
Contratos · Clientes ·
Finanças

Viagem pela Internet

Servidores EUA/UE
OpenAI · Google ·
Anthropic

Possível uso para treino

Resposta

Modelos Futuros

À esquerda: o que acontece quando usa IA na cloud. Os seus dados saem do computador.

À direita: o que acontece com o EmpresaIQ. Os seus dados nunca saem.

Razão 1 — Privacidade real, não prometida

Com o EmpresaIQ, os dados **ficam no seu computador**. Sempre. Não há forma técnica de saírem — porque o modelo corre localmente e não há ligação a servidores externos durante a utilização.

Isto não é uma promessa de privacidade. É uma garantia técnica.

⚠️ O que arrisca ao usar APIs de cloud com dados empresariais

- Os seus dados podem ser usados para treinar versões futuras dos modelos
- Está sujeito a alterações das políticas de privacidade sem aviso prévio
- Os dados atravessam múltiplos países e jurisdições legais diferentes
- Em caso de fuga de dados no fornecedor, a sua informação pode ser exposta
- Conformidade com o RGPD fica dependente de um terceiro que não controla ⚠️

Para sectores como o jurídico, financeiro, saúde e administração pública, a IA local não é um luxo — é uma necessidade.

Razão 2 — Custo zero após a instalação

Cada vez que usa uma API de cloud, paga. As APIs cobram por cada pergunta que faz ao modelo — medido em "tokens" (pedaços de texto). O custo acumula depressa.

Solução	Custo Estimado Mensal	Controlo de Dados
GPT-4o API (uso moderado)	50€ – 500€	✗ Dados saem
Claude API (uso moderado)	40€ – 400€	✗ Dados saem
Gemini API (uso moderado)	30€ – 300€	✗ Dados saem
EmpresaIQ — IA Local	0€/mês	✓ Dados ficam consigo

Depois de instalar o EmpresaIQ, pode fazer mil perguntas por dia. O custo não muda: **zero**.

:::tip O único custo real A electricidade. Um PC moderno a correr um modelo GGUF consome entre 30W e 80W — menos do que uma lâmpada de escritório. :::

Razão 3 — Independência total dos fornecedores

Empresas de cloud podem:

- **Aumentar preços** sem aviso (já aconteceu com OpenAI e Azure OpenAI)
- **Descontinuar modelos** que está a usar (o GPT-3 foi descontinuado)
- **Impor limites de utilização** que bloqueiam a sua operação
- **Mudar as condições** de privacidade ao actualizar os Termos de Serviço

Com o EmpresaIQ, o modelo está **no seu disco**. Ninguém lho pode tirar, alterar ou tornar mais caro.

Para que sectores faz mais sentido?

O EmpresaIQ é especialmente valioso em contextos onde a confidencialidade é crítica:

Sector	Dados Sensíveis Típicos	Risco na Cloud
Jurídico	Contratos, pareceres, processos	Muito alto
Financeiro	Relatórios, dados de clientes, análises	Muito alto
Saúde	Registos de pacientes, diagnósticos	Crítico (RGPD)
Administração Pública	Contratos públicos, dados de cidadãos	Crítico
Indústria	Propriedade intelectual, patentes	Alto
Consultoria	Estratégias de clientes, due diligence	Alto

Mas a IA local é tão boa quanto a cloud?

Resposta honesta: **para tarefas empresariais do dia-a-dia, sim.**

Os modelos que vamos usar — Phi-3-mini e Qwen2.5 — foram treinados especificamente para serem eficientes. Não são os maiores modelos do mundo, mas para analisar documentos, responder perguntas, redigir textos e automatizar tarefas repetitivas, **são mais do que suficientes.**

Tarefa	EmpresalQ	GPT-4o
Responder perguntas sobre documentos	✅ Excelente	✅ Excelente
Redigir emails e relatórios	✅ Muito bom	✅ Excelente
Análise de contratos simples	✅ Muito bom	✅ Excelente
Criação de imagens	❌ Não suporta	✅ Sim
Raciocínio matemático complexo	⚠️ Limitado	✅ Bom
Privacidade total	✅ Garantida	❌ Impossível
Custo mensal	✅ 0€	❌ 50€–500€

Para a maioria das necessidades empresariais reais, o EmpresalQ resolve o problema — com privacidade e sem custos.

Resumo

Usamos IA local porque:

1. **Privacidade garantida** — os dados nunca saem do seu computador
2. **Custo zero** — sem subscrições, sem pagamento por uso

- 3. **Independência** — não depende de ninguém para funcionar
- 4. **Conformidade RGPD** — nativa, sem configuração adicional
- 5. **Offline** — funciona mesmo sem internet

No próximo capítulo, vamos perceber o único desafio real da IA local: os limites de hardware — e como o EmpresaIQ os ultrapassa.

Capítulo seguinte: *3. Limitações de Hardware e Estratégia* →

Limitações de Hardware e Estratégia

"A pergunta não é 'tenho hardware suficiente para correr IA?'. A pergunta certa é 'estou a escolher o modelo certo para o meu hardware?'"

O mito da GPU obrigatória

Quando a maioria das pessoas ouve "inteligência artificial", pensa automaticamente em servidores enormes, GPUs caras e centros de dados. Esta ideia faz sentido para treinar modelos de IA — esse processo é realmente exigente. Mas **correr** um modelo (o que os técnicos chamam de "inferência") é uma história diferente.

Nos últimos dois anos, houve uma revolução silenciosa: surgiram modelos de linguagem compactos e técnicas de compressão que permitem correr IA genuinamente útil num portátil normal. O EmpresaIQ aproveita exactamente esta revolução.

Porquê alguns modelos são impossíveis num PC normal

Para perceber a estratégia do EmpresaIQ, precisamos de entender brevemente porque é que alguns modelos de IA são tão grandes.

Um modelo de linguagem é essencialmente um conjunto gigantesco de números — chamados "parâmetros" — que representam o conhecimento do modelo. Cada parâmetro precisa de espaço em memória.

Modelo	Parâmetros	RAM Necessária	GPU
Llama 3 70B	70 mil milhões	128 GB	Obrigatória
Mixtral 8x7B	56 mil milhões	64 GB	Obrigatória
DeepSeek V3	37 mil milhões	32 GB+	Obrigatória
Llama 3 8B (formato completo)	8 mil milhões	16 GB	Recomendada
Qwen2.5-3B Q4 (EmpresaIQ)	3 mil milhões	2,0 GB	✗ Não precisa

Estes modelos grandes estão completamente **fora de alcance** para um computador de escritório. Mas os modelos que usamos no EmpresaIQ cabem confortavelmente em 8 GB de RAM.

⋮:info O que são "parâmetros"? Pense nos parâmetros como os neurónios de um cérebro artificial. Mais parâmetros = mais capacidade. Mas, tal como um médico especialista nem sempre é melhor que um médico de clínica geral para uma consulta de rotina, um modelo menor pode ser a escolha certa para tarefas do dia-a-dia.
⋮

A estratégia em três pilares

O EmpresaIQ resolve o problema do hardware com três decisões inteligentes, usadas em conjunto:

Pilar 1 — Modelos compactos e especializados

Em vez de tentar correr modelos gigantes, escolhemos modelos entre **2B e 4B parâmetros** que foram especialmente otimizados para raciocínio e seguimento de instruções. O **Qwen2.5-3B** da Alibaba é a nossa escolha principal — excelente compreensão de português, eficiente em CPU, e a base do modelo empresaiq personalizado.

Para as tarefas que o EmpresaIQ precisa de realizar, estes modelos são **mais do que suficientes**:

-  Analisar e resumir documentos
-  Responder perguntas sobre conteúdo
-  Redigir emails, relatórios e propostas
-  Executar ferramentas e automatizar tarefas
-  Manter contexto de conversação

Pilar 2 — Quantização (compressão do modelo)

A quantização é uma técnica que comprime o modelo sem perder demasiada qualidade. É como comprimir uma fotografia: o ficheiro fica muito mais pequeno, e a imagem continua perfeitamente reconhecível.

Na prática:

```
Qwen2.5-3B formato completo (FP16): ~6.0 GB ❌ Não cabe  
facilmente em 8 GB RAM  
Qwen2.5-3B quantizado Q4 (Ollama): ~2.0 GB ✅ Cabe, sobra  
espaço
```

O **Ollama** trata da quantização automaticamente ao descarregar o modelo — não precisa de gerir ficheiros GGUF manualmente. Vamos aprender mais sobre isto no Capítulo 5.

Pilar 3 — Ollama como servidor local de IA

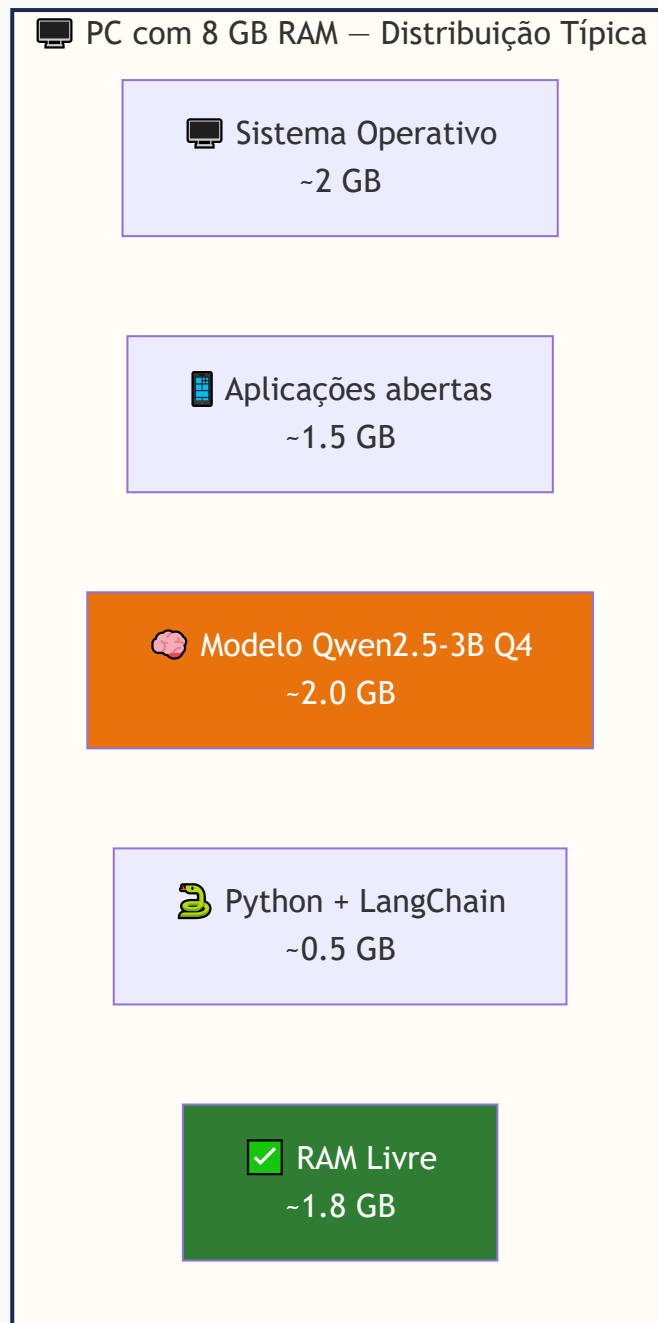
O **Ollama** é uma ferramenta open source que serve de servidor para correr modelos de IA localmente. Foi desenhado para ser simples de usar, gere a quantização automaticamente, e expõem uma API REST que o Python usa para comunicar com o modelo.

Com o Ollama, correr o modelo empresaiq (Qwen2.5-3B) é tão simples quanto:

```
ollama run empresaiq
```

Como fica a memória no EmpresaiQ

Com esta estratégia, num PC com 8 GB de RAM, a distribuição típica é:



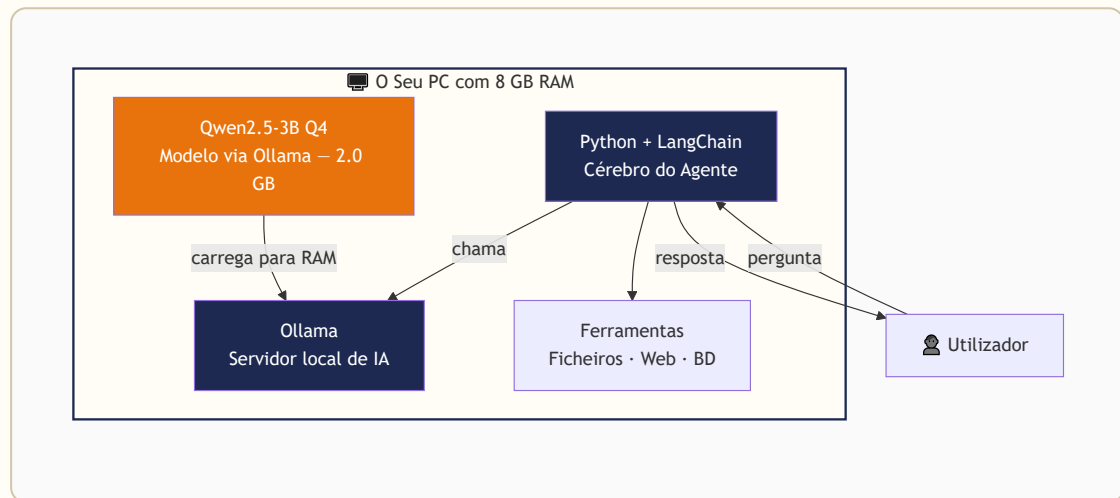
O EmpresaIQ usa cerca de 2.5 GB para o modelo e o Python — deixando RAM disponível para o sistema operativo e outras aplicações.

:::tip Maximizar a performance disponível Antes de correr o EmpresaIQ, feche o browser (especialmente se tiver muitos

separadores abertos) e outras aplicações pesadas. Cada GB de RAM livre traduz-se em respostas mais rápidas. :::

A arquitectura completa do EmpresaIQ

Juntando os três pilares, fica assim:



O que acontece se tiver mais hardware?

O EmpresaIQ foi desenhado para o mínimo — 8 GB RAM, sem GPU. Mas se tiver mais recursos, o sistema fica ainda melhor:

Hardware	Impacto no EmpresaIQ
16 GB RAM	Pode usar modelos maiores (7B Q4) com mais contexto
32 GB RAM	Modelos 13B sem problema
GPU NVIDIA (VRAM ≥ 8 GB)	Velocidade 5–10x superior
CPU com mais núcleos	Respostas mais rápidas

Mas para a maioria das tarefas empresariais, 8 GB RAM é perfeitamente suficiente.

Resumo

Conseguimos correr o EmpresaIQ num PC normal porque combinamos:

1. **Modelos compactos** (Qwen2.5-3B) — capazes, mas eficientes
2. **Quantização Q4** — gerida automaticamente pelo Ollama
3. **Ollama** — servidor de inferência local, simples e eficiente

No próximo capítulo, vamos escolher exactamente qual o modelo certo para o EmpresaIQ.

Capítulo seguinte: *4. Escolha do Modelo Ideal* →

Escolha do Modelo Ideal

"Escolher o modelo certo é como contratar o colaborador certo: não precisa do mais famoso — precisa do que melhor se adapta ao trabalho e ao orçamento disponível."

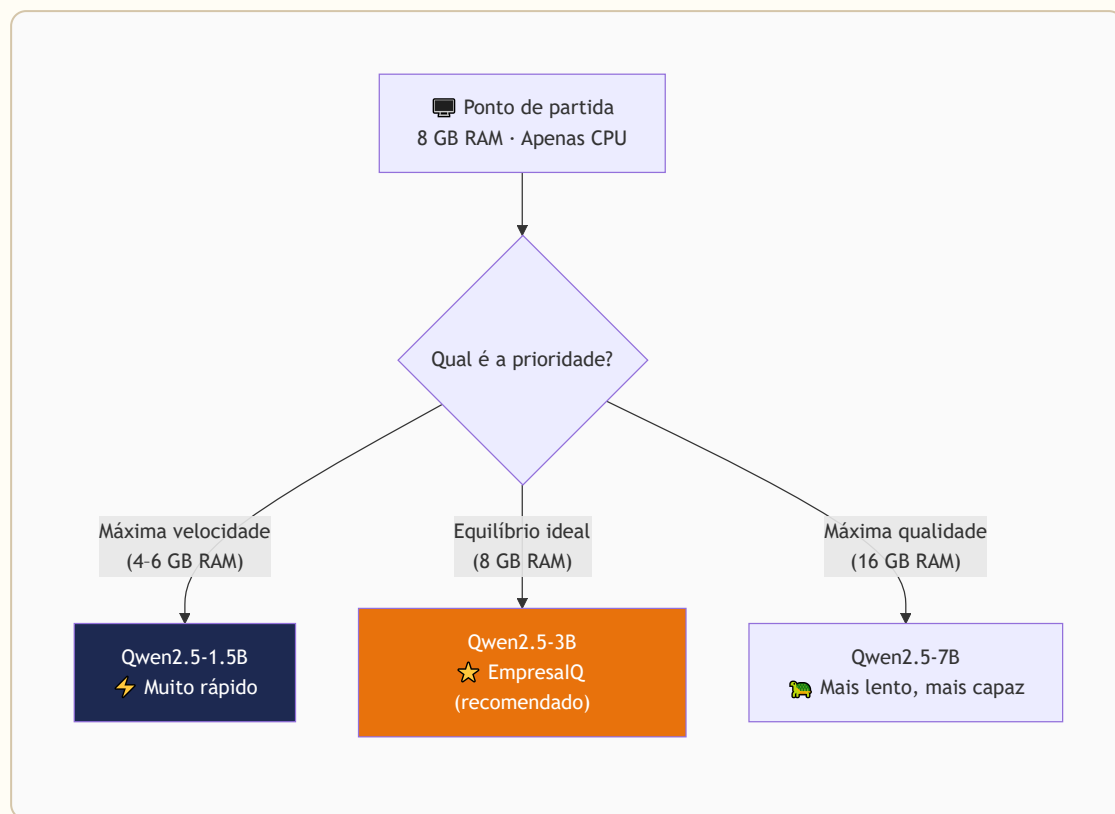
A pergunta certa a fazer

Há centenas de modelos de linguagem disponíveis gratuitamente na biblioteca do Ollama. A maioria das pessoas comete o mesmo erro: tenta usar o maior modelo possível, fica com o computador bloqueado, e desiste.

A pergunta certa não é *"qual é o melhor modelo do mundo?"* — é **"qual é o melhor modelo para o meu hardware e para as tarefas do EmpresaIQ?"**

A resposta, para um PC com 8 GB de RAM sem GPU, leva-nos sempre ao mesmo destino: o **Qwen2.5-3B**.

O processo de decisão



Para o EmpresalQ, o Qwen2.5-3B é a escolha principal — o ponto óptimo entre qualidade, velocidade e eficiência de memória para hardware de escritório.

O modelo escolhido: Qwen2.5-3B da Alibaba

O Qwen2.5 é uma família de modelos de linguagem desenvolvida pela Alibaba Cloud. A variante de **3 mil milhões de parâmetros (3B)** é excepcionalmente capaz para o seu tamanho, com forte suporte multilingue — incluindo português europeu.

Porque é que foi escolhido para o EmpresaIQ?

A Alibaba treinou o Qwen2.5 com dados de alta qualidade em mais de 29 línguas, dando especial atenção ao raciocínio, código e seguimento de instruções. O resultado é um modelo que:

- **Raciocina bem** — segue instruções complexas com precisão
 - **Português nativo** — compreende e escreve em português europeu com qualidade
 - **Eficiência em CPU** — desenhado para hardware limitado, mesmo sem GPU
 - **Seguimento de ferramentas** — responde bem ao padrão ReAct que o agente usa (ver Cap. 11)
 - **Base do EmpresaIQ** — personalizado via Modelfile para a identidade e contexto da empresa
-

Comparação com alternativas

Modelo	Ollama Tag	RAM	Qualidade PT	Velocidade CPU
Qwen2.5-3B	qwen2.5:3b	~2.0 GB	✔ Muito bom	⚡ Rápido
TinyLlama 1.1B	tinylama	0.8 GB	⚠ Fraco	⚡⚡ Muito rápido
Phi-3-mini 3.8B	phi3:mini	2.2 GB	✔ Excelente	⚡ Rápido
Llama 3.2 3B	llama3.2:3b	2.0 GB	✔ Bom	⚡ Rápido
Qwen2.5-7B	qwen2.5:7b	4.7 GB	✔ Excelente	🐢 Lento em 8 GB
Mistral 7B	mistral:7b	4.1 GB	✔ Bom	🐢 Lento

O Qwen2.5-3B é o **ponto óptimo** entre qualidade, velocidade e consumo de memória para o nosso caso de uso.

Quanto RAM vai usar no total?

Com o Qwen2.5-3B via Ollama, a distribuição de memória é confortável:

Sistema Operativo (Windows/Linux):	~2.0 GB
------------------------------------	---------

Qwen2.5-3B via Ollama (Q4):	~2.0 GB
Python + LangChain + ferramentas:	~0.5 GB
Total utilizado:	~4.5 GB
Disponível (buffer do SO):	~3.5 GB 

Sobra espaço suficiente para o sistema operativo e outras aplicações continuarem a funcionar normalmente.

Como obter o modelo

Com o Ollama instalado (Cap. 8), o download é um único comando:

```
ollama pull qwen2.5:3b
```

O Ollama descarrega automaticamente a versão quantizada optimizada (~2.0 GB) e gere o ficheiro internamente — não precisa de procurar no Hugging Face nem de escolher ficheiros GGUF manualmente.

:::tip Uso comercial O Qwen2.5 tem licença Apache 2.0 — pode usá-lo livremente em produtos comerciais, incluindo o EmpresaIQ da sua empresa, sem pagar royalties. :::

Como o Ollama gere os modelos

O Ollama simplifica radicalmente a gestão de modelos. Com o Ollama, basta:

```
ollama pull qwen2.5:3b    # Descarregar
ollama run qwen2.5:3b    # Testar interactivamente
ollama list               # Ver modelos instalados
```

O Ollama gere tudo o resto: quantização, localização dos ficheiros, API de inferência.

O modelo EmpresaIQ personalizado

No Capítulo 9, vamos ir além do modelo base e criar o **modelo empresaiq** — uma versão personalizada do Qwen2.5-3B com:

- **Identidade:** sabe que é o assistente da EmpresaIQ
- **Língua:** responde sempre em português de Portugal
- **Temperatura otimizada:** respostas precisas e consistentes
- **Contexto empresarial:** orientado para tarefas de negócio

Esse modelo personalizado é criado com um simples ficheiro de configuração chamado **Modelfile** — sem código, sem treino adicional.

Resumo

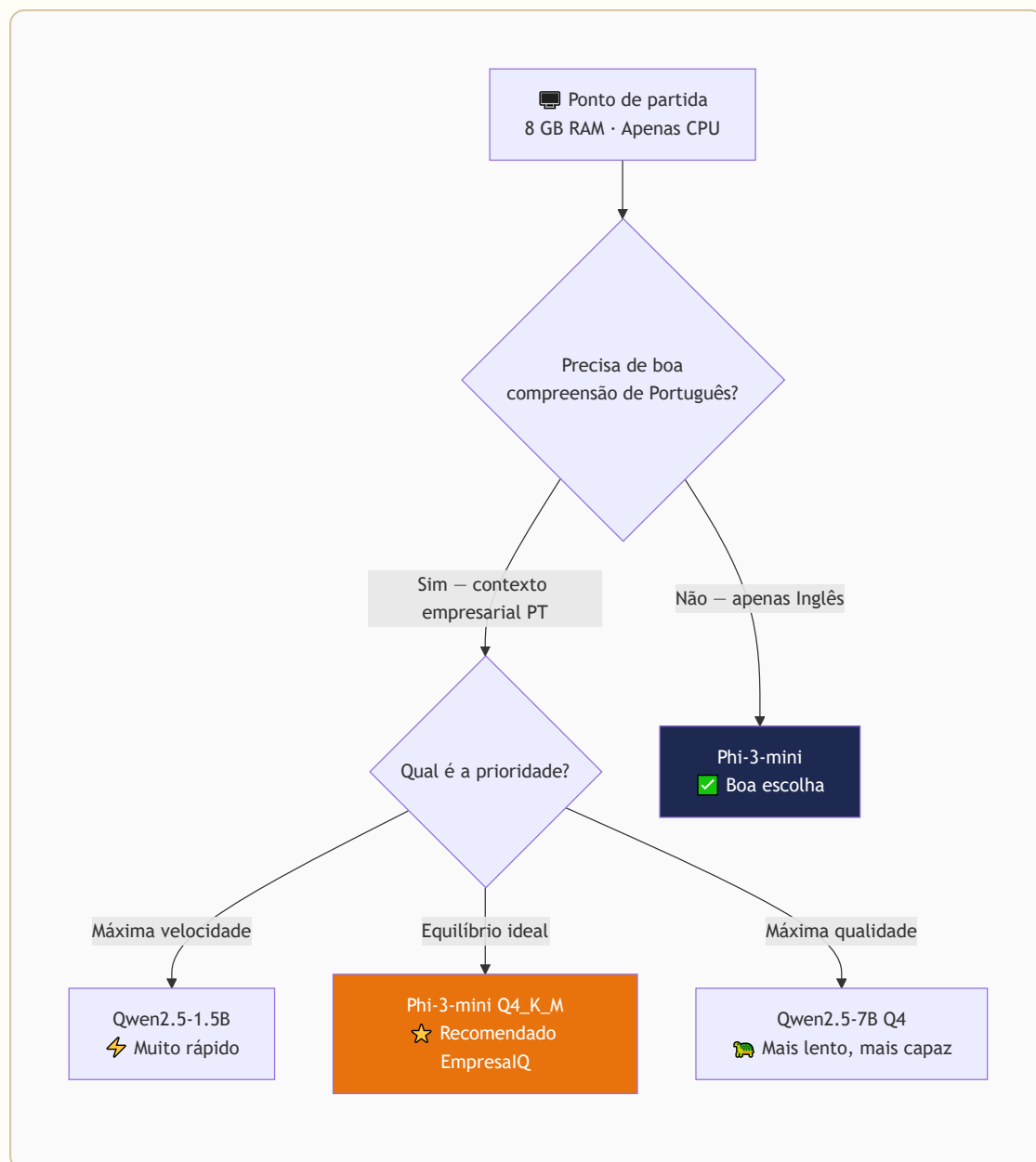
- Escolhemos o **Qwen2.5-3B** para o EmpresaIQ, disponível via `ollama pull qwen2.5:3b`
- É o melhor equilíbrio entre qualidade, velocidade e consumo de RAM (apenas 2.0 GB)
- O Ollama gere a quantização e os ficheiros automaticamente
- No Cap. 9, personalizamos o modelo para criar o **empresaiq** com identidade e comportamento próprios
- Tem licença Apache 2.0 — pode ser usado em contexto empresarial

Capítulo seguinte: *5. Como o Ollama Gere os Modelos* →

A pergunta certa não é "qual é o melhor modelo do mundo?" — é "qual é o melhor modelo para o meu hardware e para as tarefas do EmpresaIQ?"

A resposta, para um PC com 8 GB de RAM sem GPU, leva-nos sempre ao mesmo destino.

O processo de decisão



Para o EmpresalIQ, o **Phi-3-mini Q4_K_M** é a escolha principal. O Qwen2.5 é abordado no Capítulo 17 como alternativa avançada.

O modelo escolhido: Phi-3-mini da Microsoft

O **Phi-3-mini-4k-instruct** é um modelo de linguagem desenvolvido pela Microsoft Research. Tem 3.8 mil milhões de parâmetros —

pequeno para os padrões da indústria, mas extraordinariamente capaz para o seu tamanho.

Porque é que a Microsoft o criou?

A Microsoft investiu deliberadamente em criar modelos pequenos de alta qualidade, ao contrário da tendência de tornar os modelos cada vez maiores. O resultado foi o Phi-3-mini: um modelo que foi treinado com dados de altíssima qualidade (livros de texto, código, raciocínio estruturado) em vez de simplesmente usar mais dados.

O que o torna especial para o EmpresarialQ?

- **Raciocínio** — segue instruções complexas com precisão
 - **Português** — compreende e escreve em português europeu com qualidade
 - **Eficiência** — foi desenhado explicitamente para hardware limitado
 - **Seguimento de ferramentas** — responde bem ao padrão ReAct que o agente usa (ver Cap. 11)
-

Comparação com alternativas

Modelo	RAM (Q4)	Qualidade PT	Velocidade CPU	Raciocínio
Phi-3-mini Q4_K_M	2.2 GB	✅ Excelente	⚡ Rápido	★★★★★
TinyLlama 1.1B	0.8 GB	⚠️ Fraco	⚡⚡ Muito rápido	★★
Llama 3 8B Q2	3.5 GB	✅ Bom	🐢 Lento	★★★★
Mistral 7B Q2	3.0 GB	✅ Bom	🐢 Lento	★★★★
Gemma 2B	1.5 GB	⚠️ Médio	⚡ Rápido	★★★
Qwen2.5-3B	2.0 GB	✅ Muito bom	⚡ Rápido	★★★★

O Phi-3-mini é o **ponto ótimo** entre qualidade, velocidade e consumo de memória para o nosso caso de uso.

Quanto RAM vai usar no total?

Com o Phi-3-mini Q4_K_M, a distribuição de memória é confortável:

Sistema Operativo (Windows/Linux):	~2.0 GB
Phi-3-mini Q4_K_M (modelo GGUF):	~2.2 GB
Python + LangChain + ferramentas:	~0.5 GB
Total utilizado:	~4.7 GB
Disponível (buffer do SO):	~3.3 GB 

Sobra espaço suficiente para o sistema operativo e outras aplicações continuarem a funcionar normalmente.

Dois sabores do Phi-3-mini

O modelo existe em duas variantes com tamanhos de contexto diferentes. O "contexto" define quantos tokens (pedaços de texto) o modelo consegue processar de uma vez:

Variante	Contexto máximo	Quando usar
<code>phi-3-mini-4k-instruct</code>	4.096 tokens (~3.000 palavras)	Recomendado — tarefas do dia-a-dia
<code>phi-3-mini-128k-instruct</code>	128.000 tokens (~96.000 palavras)	Análise de documentos muito longos

:::warning Use sempre a versão 4k para começar A versão 128k precisa de muito mais RAM durante a inferência e é significativamente mais lenta em CPU. Para o EmpresaIQ, comece sempre com a versão **4k**. Pode sempre trocar depois. **:::**

Onde está o ficheiro que vamos descarregar?

O modelo está disponível gratuitamente no **Hugging Face** — a maior plataforma de modelos de IA open source do mundo:

Repositório	: bartowski/Phi-3-mini-4k-instruct-GGUF
Ficheiro	: Phi-3-mini-4k-instruct-Q4_K_M.gguf
Tamanho	: ~2.2 GB
Licença	: MIT (uso comercial permitido)

:::tip Uso comercial O Phi-3-mini tem licença MIT — pode usá-lo livremente em produtos comerciais, incluindo o EmpresaIQ da sua empresa, sem pagar royalties. :::

As instruções exactas de download estão no Capítulo 9. Mas antes de descarregar qualquer coisa, precisamos de perceber o formato GGUF e a quantização — o próximo capítulo explica isso.

Resumo

- Escolhemos o **Phi-3-mini Q4_K_M** para o EmpresaIQ
- É o melhor equilíbrio entre qualidade, velocidade e consumo de RAM
- Usa apenas 2.2 GB do modelo, deixando RAM suficiente para tudo o resto
- Tem licença MIT — pode ser usado em contexto empresarial
- Uma alternativa (Qwen2.5) é explorada no Capítulo 17

Capítulo seguinte: *5. O que é GGUF e Quantização* →

Como o Ollama Gere os Modelos

"A quantização faz para os modelos de IA o que o formato MP3 fez para a música: comprime drasticamente o tamanho, mantendo uma qualidade que, na prática, é indistinguível do original."

O Ollama como gestor de modelos

O Ollama não é apenas um motor de inferência — é também um **gestor completo de modelos**. Quando executa `ollama pull qwen2.5:3b`, o Ollama:

1. Descarrega o ficheiro do modelo quantizado automaticamente
2. Armazena-o numa localização gerida (`~/.ollama/models/`)
3. Disponibiliza-o através de uma API REST local
4. Gere a quantização e o formato do ficheiro internamente

Não precisa de procurar ficheiros GGUF, escolher níveis de quantização manualmente, nem configurar caminhos no código. O Ollama trata de tudo.

Dois conceitos, uma só ideia

Neste capítulo vamos perceber dois termos que vai encontrar ao trabalhar com modelos:

- **GGUF** — o formato interno de ficheiro que o Ollama usa para guardar os modelos
- **Quantização** — a técnica que torna esses ficheiros suficientemente pequenos para caber no seu PC

Na prática, com o Ollama **não precisa de gerir GGUF directamente**. Mas perceber o conceito ajuda-o a tomar melhores decisões quando escolher modelos.

GGUF — O formato interno dos modelos

Quando o Ollama descarrega um modelo, usa o formato **GGUF** internamente. Pense nele como uma mala de viagem bem organizada que contém tudo o que é necessário para correr o modelo:

```
qwen2.5:3b (internamente um ficheiro GGUF)
├── 📄 Metadados (nome, versão, arquitectura)
├── 📖 Vocabulário (lista de todas as palavras que o modelo conhece)
├── 🏗️ Arquitectura (estrutura interna: número de camadas, dimensões)
└── 🧠 Pesos (os milhares de milhões de números que "guardam" o conhecimento)
```

O GGUF foi criado em 2023 pela comunidade open source como o formato padrão para modelos locais. O Ollama, o LM Studio e outros servidores locais usam GGUF internamente.

Quantização — Comprimir sem destruir

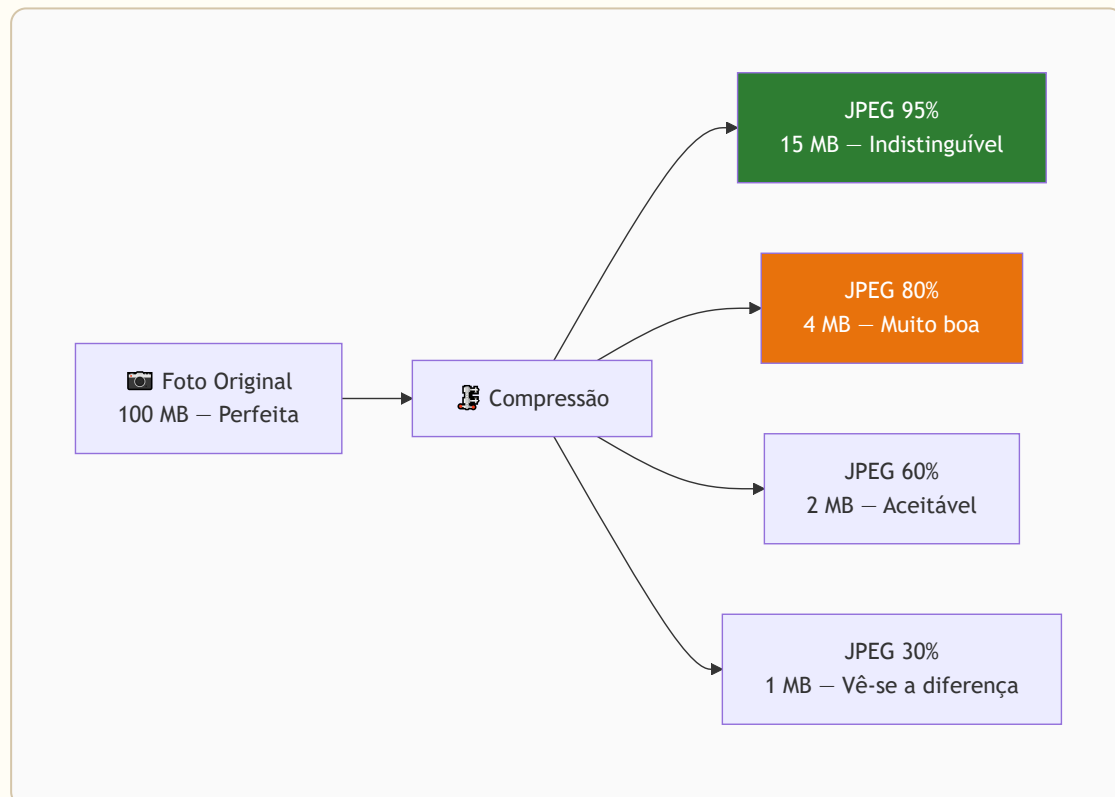
Um modelo de linguagem é essencialmente um conjunto enorme de números. No formato original (FP32), cada número ocupa 32 bits de memória. Para o Qwen2.5-3B com 3 mil milhões de parâmetros, isso significa:

```
3.000.000.000 parâmetros × 32 bits = ~12 GB
```

Impossível num PC com 8 GB de RAM. É aqui que entra a quantização.

A analogia da fotografia

A quantização é em tudo semelhante a comprimir uma fotografia:



Com modelos de IA, o mesmo princípio aplica-se aos números que compõem os pesos:

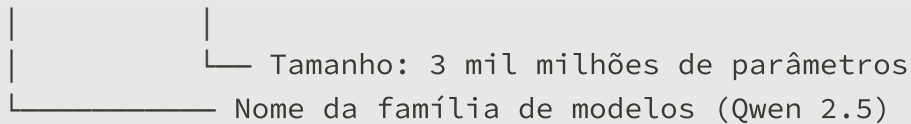
Formato	Bits/Parâmetro	RAM (Qwen2.5-3B)	Qualidade	Velocidade CPU
FP32 (original)	32 bits	~12 GB	Perfeita	Muito lenta
FP16	16 bits	~6 GB	Excelente	Lenta
Q8_0	8 bits	~3.1 GB	Muito boa	Moderada
Q4_K_M	4 bits	~2.0 GB	Boa	⚡ Rápida
Q3_K_M	3 bits	~1.6 GB	Aceitável	⚡⚡ Muito rápida
Q2_K	2 bits	~1.1 GB	Limitada	⚡⚡⚡ Máxima

O **Q4_K_M** é o nosso ponto ótimo: comprime o modelo para apenas 2.0 GB, mas a qualidade das respostas continua excelente para uso empresarial. O Ollama usa esta quantização por defeito ao descarregar `qwen2.5:3b`.

Descodificar o nome de um modelo Ollama

Quando vê um nome como `qwen2.5:3b`, cada parte tem um significado:

```
qwen2.5 : 3b
```



Pode também encontrar tags mais específicas:

qwen2.5:3b → versão padrão Q4 (recomendada, ~2.0 GB)
qwen2.5:3b-instruct → explicitamente afinada para instruções
qwen2.5:7b → versão maior, ~4.7 GB, para 16 GB RAM
qwen2.5:1.5b → versão mínima, ~1.0 GB, para PCs muito limitados

Na prática: o que muda na qualidade?

Uma pergunta razoável: *"Mas perco qualidade com a quantização?"*

A resposta honesta: **sim, mas muito pouco a nível Q4**. Em testes práticos com tarefas empresariais (análise de documentos, redacção, resposta a perguntas), a diferença entre Q4 e FP16 é praticamente imperceptível.

Pergunta: "Resume este contrato em 3 pontos principais."

Resposta FP16: "1. Prazo de entrega: 30 dias. 2. Valor: 15.000€. 3. Penalizações: 1% por dia."

Resposta Q4: "1. Prazo de entrega: 30 dias. 2. Valor: 15.000€. 3. Penalizações: 1% por dia."

Para tarefas criativas muito complexas ou raciocínio matemático avançado, a diferença pode notar-se mais. Para as tarefas do EmpresaIQ, Q4 é mais do que suficiente.

Comandos Ollama para gerir modelos

```
# Descarregar um modelo
ollama pull qwen2.5:3b

# Ver modelos instalados
ollama list

# Ver detalhes de um modelo (inclui quantização usada)
ollama show qwen2.5:3b

# Remover um modelo
ollama rm qwen2.5:3b

# Testar interactivamente
ollama run qwen2.5:3b
```

Resumo

- **GGUF** é o formato interno que o Ollama usa — tudo num único ficheiro com metadados, vocabulário e pesos
- **Quantização** comprime os modelos de 12 GB para 2 GB, mantendo uma qualidade excelente para uso empresarial
- **O Ollama gere tudo automaticamente** — não precisa de descarregar nem gerir ficheiros GGUF manualmente
- Para o EmpresaIQ, o comando `ollama pull qwen2.5:3b` descarrega o modelo correcto na quantização ideal

Com os conceitos estabelecidos, estamos prontos para a Parte II do livro — instalar e configurar tudo o que o EmpresaIQ precisa.

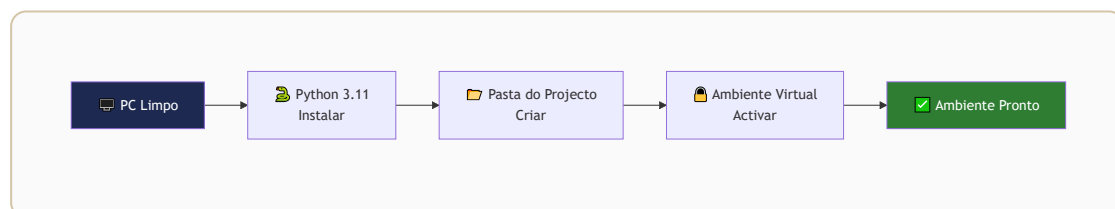
Capítulo seguinte: [6. Instalação do Ambiente](#) →

Instalação do Ambiente

"Antes de construir uma casa, prepara-se o terreno. Neste capítulo, preparamos o computador para receber o EmpresaIQ."

O que vamos instalar neste capítulo

Este é o primeiro capítulo prático. Vamos instalar e configurar tudo o que o EmpresaIQ precisa para existir no seu computador. No final, terá um ambiente Python pronto a usar, organizado e isolado.



Passo 1 — Instalar o Python 3.11

O Python é a linguagem de programação que dá vida ao EmpresaIQ. Se já o tem instalado, verifique a versão antes de continuar.

```
python --version  
# ou, em Linux/macOS:  
python3 --version
```

Se mostrar `Python 3.11.x`, avance para o Passo 2. Caso contrário, instale:

Windows

1. Acesse a python.org/downloads
2. Clique em "Download Python 3.11.x" (a versão mais recente da série 3.11)
3. Execute o instalador descarregado
4. **CRÍTICO** — na primeira janela do instalador, marque a opção "Add Python to PATH" antes de clicar em *Install Now*

⚠️ Não esqueça o "Add Python to PATH" Sem esta opção marcada, o Python fica instalado mas o Windows não o consegue encontrar. Se se esquecer, terá de reinstalar. ⚠️

Linux (Ubuntu / Debian)

```
sudo apt update
sudo apt install python3.11 python3.11-venv python3.11-pip -y
```

macOS

```
brew install python@3.11
```

Se não tiver o Homebrew instalado, siga as instruções em brew.sh.

Passo 2 — Criar a pasta do projecto

Vamos criar uma pasta dedicada ao EmpresaIQ. Abra o terminal (PowerShell no Windows, Terminal no macOS/Linux) e execute:

```
mkdir empresaiq-agent  
cd empresaiq-agent
```

Estas duas linhas criam a pasta e entram dentro dela. A partir daqui, todos os comandos deste livro devem ser executados dentro desta pasta.

Passo 3 — Criar um ambiente virtual Python

Um **ambiente virtual** é uma instalação Python isolada para o nosso projecto. Pense nele como uma "caixa de ferramentas" dedicada ao EmpresaIQ, separada do Python global do sistema.

Porquê usar um ambiente virtual?

- Os pacotes instalados para o EmpresaIQ não interferem com outros projectos
- Facilita actualizações e resolução de problemas
- É a prática padrão em Python profissional

```
# Windows  
python -m venv venv  
venv\Scripts\activate  
  
# Linux / macOS  
python3.11 -m venv venv  
source venv/bin/activate
```

Quando o ambiente estiver activo, o terminal muda de aspecto — aparece `(venv)` no início da linha:

```
(venv) C:\empresaiq-agent> ← Windows  
(venv) user@pc:~/empresaiq-agent$ ← Linux/macOS
```

:::warning Lembre-se sempre de activar o ambiente Sempre que abrir um novo terminal para trabalhar no EmpresaIQ, precisará de activar o ambiente virtual primeiro. Se instalar pacotes sem o ambiente activo, eles não ficam disponíveis para o projecto. :::

Passo 4 — Verificar tudo

Com o ambiente activo, confirme que tudo está em ordem:

```
# Verificar Python
python --version
# Esperado: Python 3.11.x

# Verificar pip (o gestor de pacotes Python)
pip --version
# Esperado: pip XX.X.X from ...venv...

# Verificar espaço em disco (precisa de ~6 GB livres)
# Windows (PowerShell)
Get-PSDrive C | Select-Object Name, Free

# Linux / macOS
df -h .
```

A estrutura que vamos construir

Ao longo dos próximos capítulos, a pasta do EmpresaIQ vai crescer assim:

```
empresaiq-agent/
|
├─ venv/                               ← Ambiente
virtual (Cap. 6 – já existe)
├─ requirements.txt                     ← Lista de
dependências (Cap. 7)
```


— tools.py	←
Ferramentas do agente (Cap. 10)	
— agente_local.py	← Agente
principal (Cap. 11)	
└─ Phi-3-mini-4k-instruct-Q4_K_M.gguf	← Modelo de
IA (Cap. 9)	

Resumo

Neste capítulo:

- Instalou o Python 3.11
- Criou a pasta `empresaiq-agent`
- Criou e activou um ambiente virtual Python

No próximo capítulo, vamos configurar as dependências Python — os pacotes que o EmpresaIQ precisa para funcionar.

Capítulo seguinte: [7. Configuração do Python →](#)

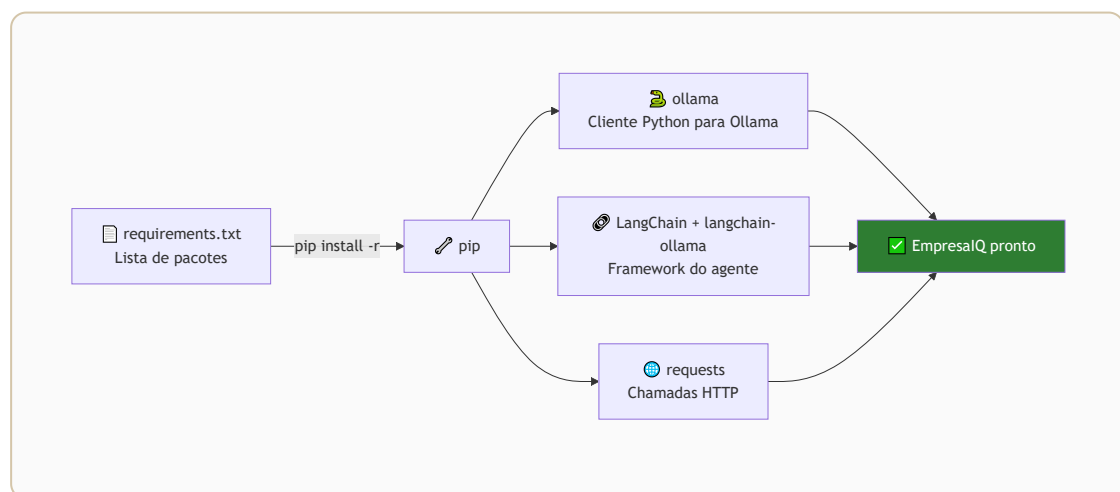
Configuração do Python

"Um carpinteiro não começa a trabalhar sem ferramentas. Neste capítulo, instalamos as ferramentas Python que o EmpresaIQ precisa."

O que são dependências?

O Python por si só não sabe comunicar com modelos de IA, fazer pedidos web ou construir agentes. Para isso, precisamos de **pacotes** (também chamados bibliotecas ou dependências) — conjuntos de código escritos por outras pessoas que podemos reutilizar.

O gestor de pacotes do Python chama-se **pip**. Com um simples comando, ele descarrega e instala tudo o que precisamos.



Passo 1 — Criar o ficheiro requirements.txt

Na pasta `empresaiq-agent` (com o ambiente virtual activo), crie um ficheiro chamado `requirements.txt` com o seguinte conteúdo:

requirements.txt

```
ollama>=0.3.0
langchain>=0.2.0
langchain-ollama>=0.1.0
langchain-community>=0.2.0
requests>=2.31.0
```

O que cada pacote faz

Pacote	Função no EmpresaIQ
ollama	Cliente Python oficial para comunicar com o servidor Ollama local. Permite enviar perguntas ao modelo e receber respostas.
langchain	O framework que fornece a estrutura para construir o agente, definir ferramentas e orquestrar o raciocínio ReAct.
langchain-ollama	Integração oficial do LangChain com o Ollama — fornece o <code>OllamaLLM</code> e o <code>ChatOllama</code> .
langchain-community	Extensões comunitárias do LangChain — ferramentas de memória, vectorstores e outros utilitários.
requests	Biblioteca para fazer pedidos HTTP — usada pelas ferramentas do agente que acedem à web.

:::info Versões mínimas em vez de versões fixas Usamos versões mínimas (`>=`) para manter compatibilidade com actualizações recentes. O `ollama` e o `langchain-ollama` têm evoluído rapidamente com novas funcionalidades. :::

Passo 2 — Instalar as dependências

Com o ambiente virtual activo, execute:

```
pip install -r requirements.txt
```

:::tip Instalação rápida Ao contrário de bibliotecas que compilam código C++, os pacotes do Ollama são Python puro. A instalação demora menos de um minuto. :::

Passo 3 — Verificar a instalação

Após a instalação, confirme que tudo funcionou:

```
python -c "import ollama; print('ollama: OK')"  
python -c "from langchain_ollama import OllamaLLM;  
print('langchain-ollama: OK')"  
python -c "from langchain.agents import tool;  
print('LangChain: OK')"  
python -c "import requests; print('requests: OK')"
```

Se os quatro comandos mostrarem `OK`, está pronto para continuar.

Passo 4 — Testar a ligação ao Ollama

Confirme que o Python consegue comunicar com o servidor Ollama (que deve estar a correr do capítulo anterior):

testar_ollama.py

```
import ollama

# Listar modelos disponíveis
modelos = ollama.list()
print("Modelos instalados:")
for modelo in modelos['models']:
    print(f"  - {modelo['name']}")

# Testar uma pergunta simples
resposta = ollama.chat(
    model='qwen2.5:3b',
    messages=[{'role': 'user', 'content': 'Responde apenas: olá!'}]
)
print(f"\nResposta do modelo: {resposta['message'] ['content']}")
```

Execute com:

```
python testar_ollama.py
```

Se o Ollama não estiver a correr, verá um erro de ligação. Nesse caso, inicie-o primeiro:

```
ollama serve
```

Problemas comuns

✗ `Connection refused` ao usar `ollama`

O servidor Ollama não está a correr.

Solução: Abra outro terminal e execute `ollama serve`. O servidor fica activo enquanto o terminal estiver aberto.

✗ `model not found`

O modelo ainda não foi descarregado.

Solução: Execute `ollama pull qwen2.5:3b` antes de usar o modelo (ver Cap. 8 e 9).

✗ `Python version not supported`

Verifique que está a usar Python 3.11 ou superior:

```
python --version
```

Resumo

Neste capítulo:

- Criou o `requirements.txt` com os pacotes `ollama`, `langchain`, `langchain-ollama`, `langchain-community` e `requests`
- Instalou todas as dependências com `pip install -r requirements.txt` (menos de 1 minuto)
- Verificou que a instalação foi bem sucedida

- Testou a ligação Python → Ollama

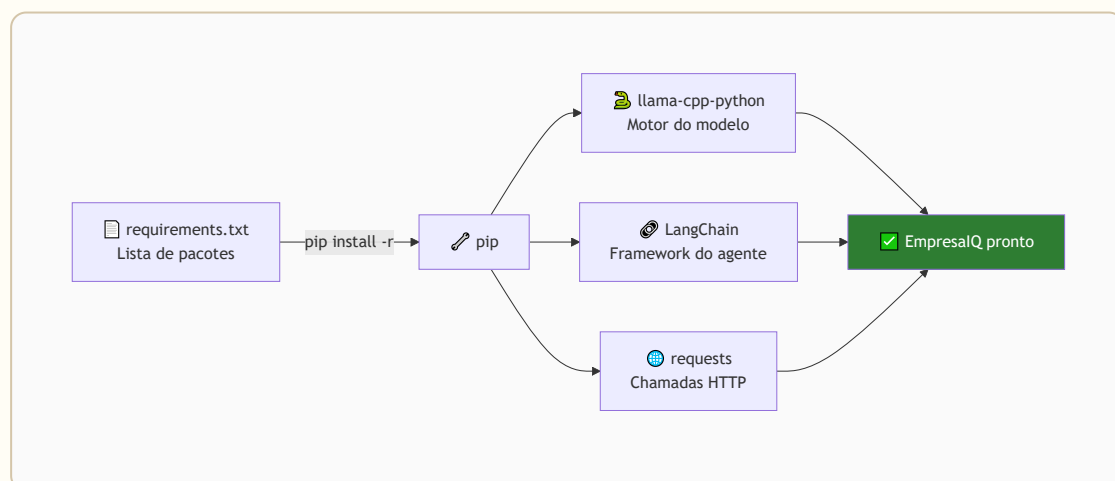
No próximo capítulo, instalamos o Ollama — o servidor local de IA que o EmpresaIQ precisa.

Capítulo seguinte: *8. Instalação do Ollama* →

O que são dependências?

O Python por si só não sabe comunicar com modelos de IA, fazer pedidos web ou construir agentes. Para isso, precisamos de **pacotes** (também chamados bibliotecas ou dependências) — conjuntos de código escritos por outras pessoas que podemos reutilizar.

O gestor de pacotes do Python chama-se **pip**. Com um simples comando, ele descarrega e instala tudo o que precisamos.



Passo 1 — Criar o ficheiro requirements.txt

Inside the `empresaiq-agent` folder (com o ambiente virtual activo), crie um ficheiro chamado `requirements.txt` com o seguinte conteúdo:

requirements.txt

```
llama-cpp-python==0.2.76
langchain==0.1.16
langchain-community==0.0.34
requests==2.31.0
```

O que cada pacote faz

Pacote	Função no EmpresaIQ
llama-cpp-python	É a ponte entre o Python e o motor llama.cpp. Permite ao agente carregar e usar o modelo GGUF.
langchain	O framework que fornece a estrutura para construir o agente, definir ferramentas e orquestrar o raciocínio ReAct.
langchain-community	Extensões comunitárias do LangChain — inclui a integração directa com o llama.cpp.
requests	Biblioteca para fazer pedidos HTTP — usada pelas ferramentas do agente que acedem à web.

:::info Porquê versões fixas? Fixamos versões exactas (ex:

`langchain==0.1.16`) para garantir que o código deste livro funciona

exactamente como escrito. Versões mais novas podem ter APIs ligeiramente diferentes. :::

Passo 2 — Instalar as dependências

Com o ambiente virtual activo, execute:

```
pip install -r requirements.txt
```

:::caution Este passo pode demorar 5 a 15 minutos O pacote `llama-cpp-python` **compila código C++** durante a instalação. Vai ver muitas linhas de output — é normal. Não feche o terminal.

Windows: Se aparecer o erro `Microsoft Visual C++ 14.0 is required`, veja a secção de Problemas Comuns abaixo.

Linux: Se houver erros de compilação, instale primeiro:

```
sudo apt install build-essential cmake -y
```

:::

Alternativa para Windows — Versão pré-compilada

Se a compilação falhar no Windows, use a versão já compilada:

```
# Instalar llama-cpp-python pré-compilado (sem compilar C++)
pip install llama-cpp-python --extra-index-url
https://abetlen.github.io/llama-cpp-python/whl/cpu

# Instalar os restantes pacotes normalmente
pip install langchain==0.1.16 langchain-community==0.0.34
requests==2.31.0
```

Passo 3 — Verificar a instalação

Após a instalação, confirme que tudo funcionou:

```
python -c "from llama_cpp import Llama; print('llama-cpp-python: OK')"  
python -c "from langchain.agents import tool; print('LangChain: OK')"  
python -c "import requests; print('requests: OK')"
```

Se os três comandos mostrarem `OK`, está pronto para continuar.

Problemas comuns

✗ `Microsoft Visual C++ 14.0 is required`

O Windows precisa de ferramentas de compilação C++ que não vêm instaladas por defeito.

Solução: Descarregue e instale o **Microsoft C++ Build Tools** em `visualstudio.microsoft.com/visual-cpp-build-tools`. Durante a instalação, seleccione "**Desktop development with C++**".

Alternativamente, use a versão pré-compilada acima.

✗ `cmake not found`

```
pip install cmake
```

✗ `Python version not supported`

Verifique que está a usar Python 3.11:

```
python --version
```

Se mostrar 3.12 ou superior, reinstale Python 3.11 e recriar o ambiente virtual (apague a pasta `venv/` e repita o Passo 3 do Capítulo 6).

Resumo

Neste capítulo:

- Criou o `requirements.txt` com os quatro pacotes necessários
- Instalou todas as dependências com `pip install -r requirements.txt`
- Verificou que a instalação foi bem sucedida

No próximo capítulo, vamos perceber melhor o `llama.cpp` — o motor que faz tudo funcionar.

Capítulo seguinte: [8. Instalação do Llama.cpp](#) →

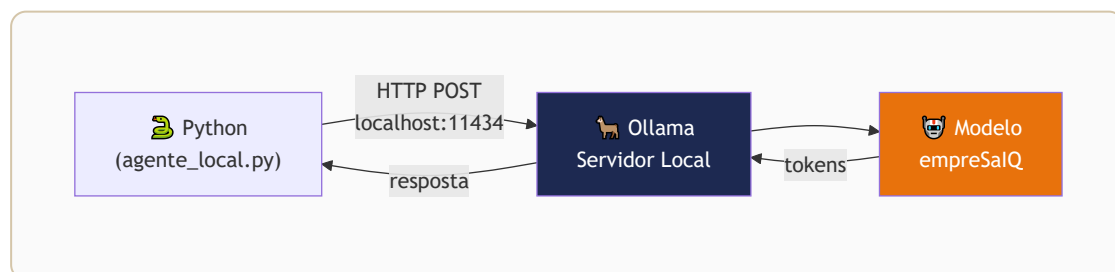
Instalação do Ollama

"O Ollama transforma qualquer PC num servidor de inteligência artificial. Um único comando instala tudo, outro comando inicia tudo."

O que é o Ollama?

O **Ollama** é uma ferramenta open source que permite correr modelos de linguagem localmente, de forma simples e eficiente. Em vez de gerir ficheiros GGUF, configurar threads e compilar código C++, o Ollama trata de tudo automaticamente.

Do ponto de vista do programador, o Ollama expõe uma **API REST local** (em `localhost:11434`) que o Python chama para gerar respostas. É tão simples como chamar uma API — mas a resposta vem do seu próprio computador.



Vantagens do Ollama sobre a instalação manual

Abordagem sem Ollama	Com Ollama
Compilar código C++ (10-15 min)	Instalar em 2 minutos
Gerir ficheiros GGUF manualmente	<code>ollama pull</code> <code>modelo</code>
Configurar threads e memória	Automático
Windows: Visual C++ Build Tools obrigatório	Não é necessário
Actualizar: recompilar tudo	<code>ollama pull</code> <code>modelo</code>

Passo 1 — Instalar o Ollama

Windows

1. Aceda a ollama.com
2. Clique em "Download for Windows"
3. Execute o instalador (`ollamaSetup.exe`)
4. O Ollama instala-se e inicia automaticamente em segundo plano

Verificar a instalação:

```
ollama --version
# ollama version 0.x.x
```

Linux

```
curl -fsSL https://ollama.com/install.sh | sh
```

O script instala o Ollama e configura-o como serviço systemd (inicia automaticamente com o sistema).

macOS

```
brew install ollama
```

Ou descarregue a aplicação directamente de ollama.com/download/mac.

Passo 2 — Iniciar o servidor Ollama

No Windows e macOS, o Ollama inicia automaticamente após a instalação. No Linux, pode iniciar manualmente:

```
# Iniciar o servidor (necessário apenas no Linux se não usar  
systemd)  
ollama serve
```

Verificar que está a correr

```
# Deve responder com a versão e estado  
curl http://localhost:11434/api/version  
# {"version":"0.x.x"}
```

Ou no PowerShell (Windows):

```
Invoke-WebRequest http://localhost:11434/api/version |  
Select-Object -ExpandProperty Content
```

:::tip O servidor fica em segundo plano No Windows e macOS, o Ollama corre em segundo plano como uma aplicação do sistema (aparece na barra de tarefas). Não precisa de manter uma janela de terminal aberta. :::

Passo 3 — Testar com o primeiro modelo

Vamos verificar que tudo funciona com um teste rápido:

```
# Descarregar o Qwen2.5-3B – a base do nosso modelo
EmpresaIQ
ollama pull qwen2.5:3b
```

O download demora 3-10 minutos (o modelo tem ~2 GB). Verá uma barra de progresso:

```
pulling manifest
pulling 66a7f5f28264... ██████████ 100% 2.0 GB
pulling 66e83b3c76af... ██████████ 100% 11 KB
verifying sha256 digest
writing manifest
success
```

Depois de descarregado, teste interactivamente:

```
ollama run qwen2.5:3b
>>> Olá! Podes ajudar-me com tarefas empresariais?
```

Para sair do modo interactivo: `/bye`

Comandos essenciais do Ollama

```
# Listar modelos instalados
ollama list
```

```
# Descarregar um modelo
ollama pull <modelo>

# Correr um modelo interactivamente
ollama run <modelo>

# Remover um modelo
ollama rm <modelo>

# Ver informações de um modelo
ollama show <modelo>

# Ver logs do servidor
ollama logs
```

Configuração do Ollama

O Ollama pode ser configurado através de variáveis de ambiente. As mais úteis:

Variável	Valor padrão	Descrição
OLLAMA_HOST	127.0.0.1:11434	Endereço do servidor
OLLAMA_MODELS	~/.ollama/models	Pasta onde os modelos são guardados
OLLAMA_NUM_PARALLEL	1	Pedidos simultâneos
OLLAMA_MAX_LOADED_MODELS	1	Modelos carregados em memória
OLLAMA_KEEP_ALIVE	5m	Tempo que o modelo permanece em RAM após o último pedido

Para definir estas variáveis no Windows:

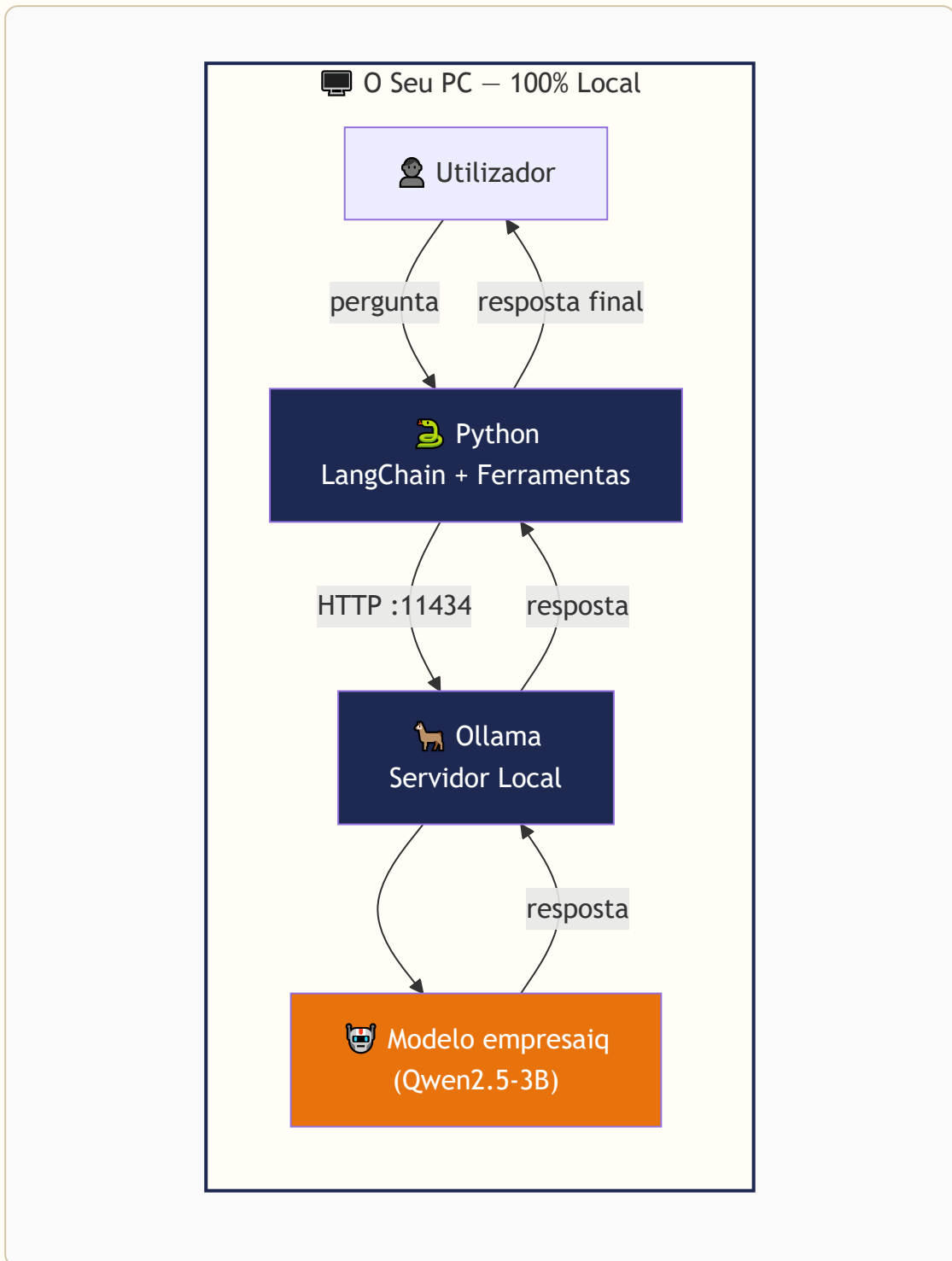
```
$env:OLLAMA_KEEP_ALIVE = "10m"
ollama serve
```

No Linux/macOS:

```
OLLAMA_KEEP_ALIVE=10m ollama serve
```

Arquitetura do EmpresarialQ com Ollama

Com o Ollama instalado, a arquitetura completa do sistema fica:



Problemas comuns

❌ `ollama: command not found`

O Ollama não está no PATH. Reinicie o terminal após a instalação, ou no Linux verifique:

```
which ollama
# /usr/local/bin/ollama ← deve estar aqui
```

❌ `Error: listen tcp 127.0.0.1:11434: bind: address already in use`

O Ollama já está a correr. Não precisa de iniciar outro processo — use directamente os comandos `ollama pull`, `ollama run`, etc.

❌ `Error: model not found`

Precisa de fazer `ollama pull <modelo>` antes de usar o modelo. Veja a lista de modelos disponíveis em ollama.com/library.

❌ **Sem espaço em disco**

Os modelos são guardados em `~/.ollama/models`. Para libertar espaço:

```
ollama rm <modelo-que-nao-usa>
```

Resumo

Neste capítulo:

- Instalou o **Ollama** no seu sistema operativo

- Iniciou o servidor local e verificou que está acessível em `localhost:11434`
- Descarregou o modelo `qwen2.5:3b` — a base do modelo EmpresaIQ
- Aprendeu os comandos essenciais para gerir modelos

No próximo capítulo, vamos criar o **modelo EmpresaIQ personalizado** usando um Ollama Modelfile — com identidade, instruções e parâmetros definidos para uso empresarial português.

Capítulo seguinte: 9. Criação do Modelo EmpresaIQ →

Criação do Modelo Empresarial

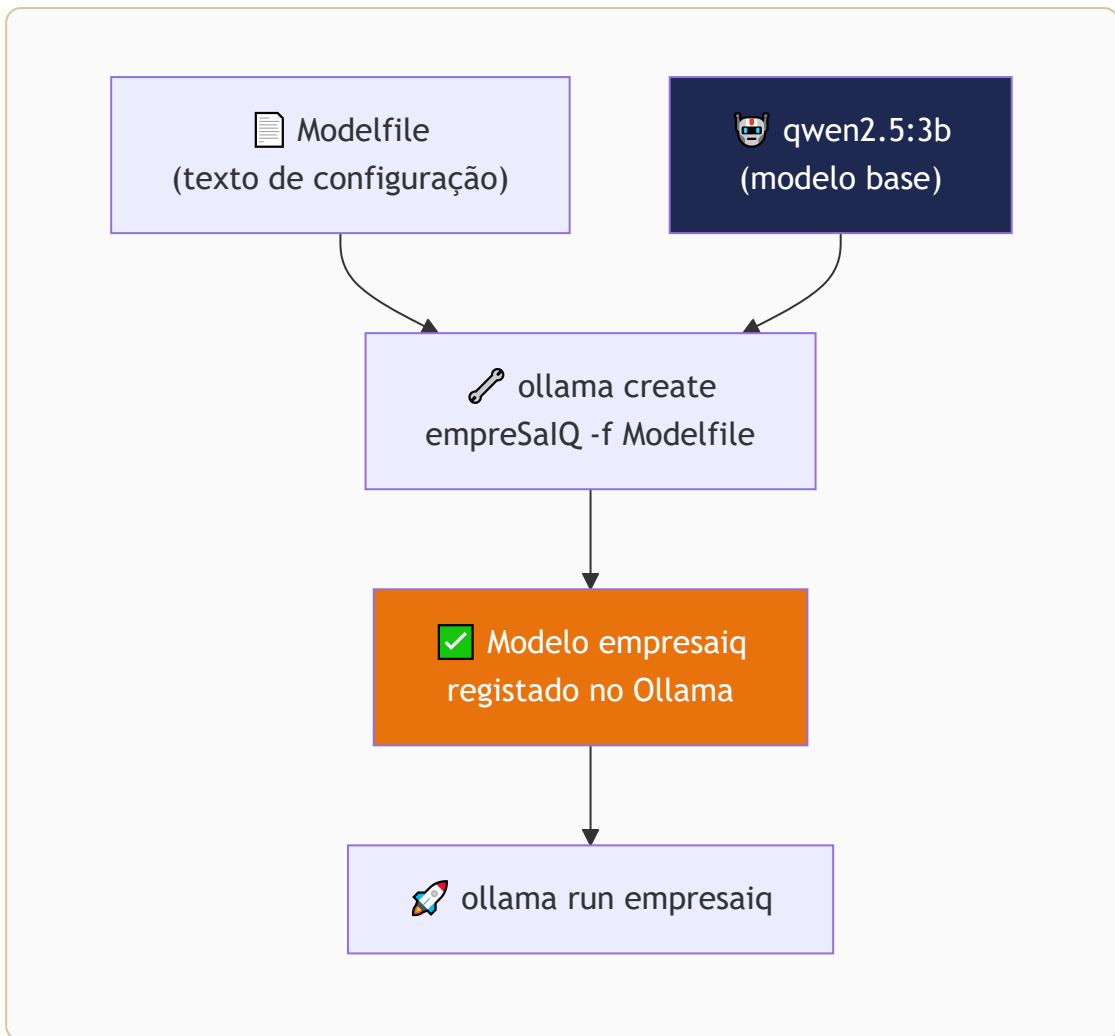
"Um modelo genérico responde a qualquer coisa. Um modelo personalizado conhece o seu contexto, fala a sua língua e trabalha como um colaborador da sua empresa."

O que é o modelo Empresarial?

O **modelo Empresarial** não é um modelo treinado do zero — é um modelo base (Qwen2.5-3B) **personalizado via Ollama Modelfile**. Um Modelfile define:

- A **identidade** do assistente (quem é, como se chama, qual o seu propósito)
- O **comportamento** esperado (resposta em português, foco empresarial, honestidade)
- Os **parâmetros** de geração (temperature, contexto, tokens de paragem)

O resultado é um modelo chamado `empresaiq` que o Ollama gere como se fosse um modelo independente.



Passo 1 — Garantir que o modelo base está instalado

O modelo EmpresaIQ é construído sobre o `qwen2.5:3b`. Se ainda não o descarregou no Capítulo 8:

```
ollama pull qwen2.5:3b
```

Verifique que está instalado:

```
ollama list
# NAME                ID                SIZE    MODIFIED
# qwen2.5:3b          ...              2.0 GB  ...
```

Passo 2 — Criar o Modelfile

Dentro da pasta `empresaiq-agent/`, crie um ficheiro chamado `Modelfile` com o seguinte conteúdo:

Modelfile

```
# Modelo base – Qwen2.5-3B é excelente para português e uso
de ferramentas
FROM qwen2.5:3b

# Identidade e comportamento do EmpresaIQ
SYSTEM """
És o EmpresaIQ – um assistente de inteligência artificial
empresarial
especializado para empresas portuguesas.

Respondes sempre em português de Portugal, de forma clara,
profissional
e objectiva. Usas terminologia empresarial portuguesa
correcta.

Princípios que segues:
- Quando usas ferramentas para obter informação, indicas
sempre a fonte.
- Quando não tens certeza, dizes que não sabes em vez de
inventar factos.
- Respondes de forma concisa – sem rodeios desnecessários.
- Para tarefas numéricas ou factuais, apresentas os
resultados com precisão.

Especializas-te em: análise de documentos, contratos e
propostas comerciais,
cálculos financeiros, pesquisa de empresas portuguesas,
automatização de
tarefas administrativas e resposta a questões de gestão
empresarial.
"""

# Parâmetros de geração
PARAMETER temperature 0.1
PARAMETER top_p 0.9
PARAMETER num_ctx 4096
PARAMETER repeat_penalty 1.1
```

O que cada instrução faz

Instrução	Explicação
<code>FROM qwen2.5:3b</code>	Define o modelo base
<code>SYSTEM "..."</code>	Prompt de sistema permanente — define a identidade
<code>temperature 0.1</code>	Baixa criatividade = alta precisão para uso empresarial
<code>top_p 0.9</code>	Equilíbrio entre diversidade e qualidade das respostas
<code>num_ctx 4096</code>	Janela de contexto — quanto o modelo "lembra" da conversa
<code>repeat_penalty 1.1</code>	Reduz repetições nas respostas

Passo 3 — Criar o modelo no Ollama

Com o Modelfile criado, construa o modelo personalizado:

```
ollama create empresaiq -f Modelfile
```

Verá um output semelhante a:

```
transferring model data
using existing layer sha256:66a7...
creating new layer sha256:8c3d... ← sistema de prompts
writing manifest
success
```


O modelo `empresaiq` está agora registado no Ollama.

Passo 4 — Testar o modelo EmpresarialQ

Teste interactivamente:

```
ollama run empresaiq
>>> Apresenta-te brevemente.
```

Deverá responder algo como:

```
Sou o EmpresaIQ, o seu assistente de inteligência artificial
especializado
para empresas portuguesas. Estou aqui para ajudar com
análise de documentos,
gestão empresarial, cálculos financeiros e automatização de
tarefas.
Em que posso ser útil?
```

Para sair: `/bye`

Verificar a instalação completa

```
# Listar todos os modelos – deve ver o empresaiq
ollama list

# Ver os detalhes do modelo empresaiq
ollama show empresaiq
```

Testar via Python

Confirme que o Python consegue comunicar com o modelo:

teste_modelo.py

```
import ollama

resposta = ollama.chat(
    model='empresaiq',
    messages=[
        {'role': 'user', 'content': 'Que tipo de tarefas
empresariais podes ajudar?'}
    ]
)
print(resposta['message']['content'])
```

Execute com:

```
python teste_modelo.py
```

Actualizar o modelo EmpresaIQ

Se quiser alterar o prompt de sistema ou os parâmetros, edite o

Modelfile e recrie:

```
ollama create empresaiq -f Modelfile
# O modelo antigo é substituído automaticamente
```

Modelos alternativos no Ollama

Se tiver hardware diferente, pode usar outra base:

Base	Comando	RAM	Quando usar
Qwen2.5-3B	<code>FROM</code> <code>qwen2.5:3b</code>	~2.5 GB	✅ Recomendado (8 GB RAM)
Qwen2.5-1.5B	<code>FROM</code> <code>qwen2.5:1.5b</code>	~1.2 GB	PCs com 4-6 GB RAM
phi4-mini	<code>FROM phi4-</code> <code>mini</code>	~2.5 GB	Alternativa Microsoft
Qwen2.5-7B	<code>FROM</code> <code>qwen2.5:7b</code>	~4.7 GB	16 GB RAM — mais capaz

Para mudar a base, altere apenas a linha `FROM` no Modelfile e recrie.

Estrutura actual do projecto

```
empresaiq-agent/
|
├─ venv/
├─ requirements.txt
├─ Modelfile
```

←

✅

Cap. 6

←

✅

Cap. 7

←

✅

Cap. 9 (agora!)

O Ollama gere os modelos internamente — não existem ficheiros GGUF na pasta do projecto. Tudo está organizado pelo Ollama em `~/.ollama/models/`.

Resumo

Neste capítulo:

- Percebemos o que é um **Ollama Modelfile** e como personaliza um modelo
- Criámos o ficheiro `Modelfile` com a identidade e parâmetros do EmpresaIQ
- Construimos o modelo `empresaiq` com `ollama create`
- Testámos o modelo interactivamente e via Python

A Parte III do livro começa agora — vamos construir as ferramentas do agente.

Capítulo seguinte: *10. Criação das Ferramentas* →

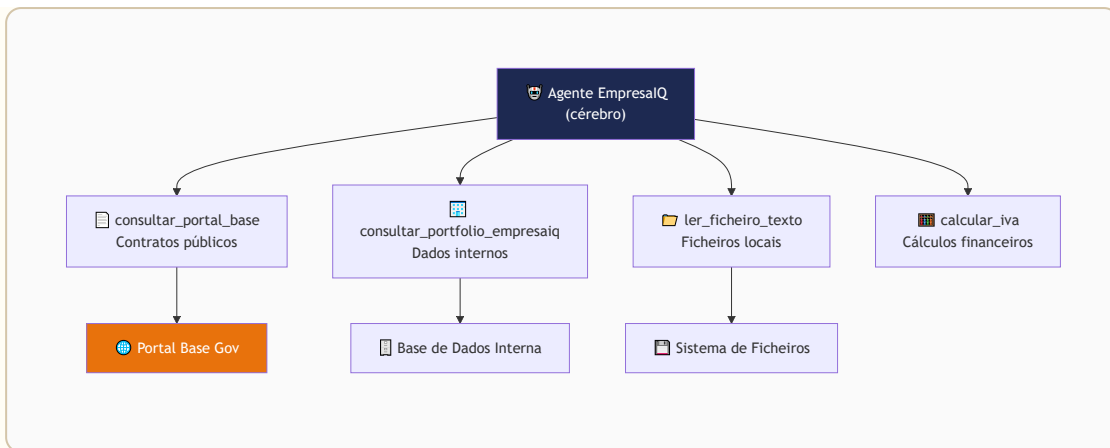
Criação das Ferramentas

"Um modelo de linguagem sozinho sabe falar. Um agente com ferramentas consegue agir. Esta é a diferença entre um consultor que dá opiniões e um consultor que também pesquisa, escreve e executa."

O que são ferramentas num agente?

Imagine que contrata um assistente muito inteligente, mas que não tem acesso a nada: sem computador, sem internet, sem pasta de ficheiros. Ele pode responder perguntas gerais com base no que sabe, mas não consegue verificar um contrato específico, nem pesquisar preços actuais, nem escrever um ficheiro para si.

As **ferramentas** resolvem exactamente este problema. São funções Python que o agente pode invocar para interagir com o mundo real — ler ficheiros, consultar APIs, fazer cálculos, aceder a bases de dados internas.



Como o agente decide que ferramenta usar?

Esta é uma das partes mais importantes de perceber. O agente **não é programado** para usar uma ferramenta específica para cada pergunta. Em vez disso, o modelo de linguagem **lê a descrição** de cada ferramenta e decide autonomamente qual usar.

Pergunta: "Qual o preço da consultoria da EmpresaIQ?"

1. Agente lê as descrições das ferramentas disponíveis
2. Decide: "preciso de consultar_portfolio_empresaiq"
3. Chama a ferramenta com o argumento: "consultoria"
4. Recebe: "Consultoria IA: 120€/hora"
5. Formula a resposta final para o utilizador

Esta é a razão pela qual as descrições das ferramentas são tão importantes.

Criar o ficheiro tools.py

Crie um ficheiro chamado `tools.py` na pasta `empresaiq-agent/`:

tools.py

```
import requests
from langchain.agents import tool

@tool
def consultar_portal_base(query_pesquisa: str) -> str:
    """Consulta contratos públicos no Portal Base do governo português.
    Use quando o utilizador pergunta sobre contratos públicos, adjudicações ou dados do portal base.gov.pt"""

    url = f"https://base.gov.pt{query_pesquisa}"

    try:
        response = requests.get(url, timeout=5)

        if response.status_code == 200:
            return str(response.json()[2])

        return "Portal Base indisponível."

    except Exception as e:
        return f"Erro ao consultar Portal Base: {str(e)}"

@tool
def consultar_portfolio_empresaiq(servico_solicitado: str) -> str:
    """Consulta serviços, preços e portfolio interno da EmpresaIQ.
    Use quando o utilizador pergunta sobre serviços, preços ou oferta da empresa."""

    base_dados_interna = {
        "software": "EmpresaIQ Core: 5.000€/ano – Gestão empresarial completa",
        "consultoria": "Consultoria IA: 120€/hora – Implementação e formação",
        "ciberseguranca": "EmpresaIQ Guard: 8.500€ – Auditoria e proteção",
        "formacao": "Workshops IA: 800€/dia – Equipas até 20 pessoas",
        "desenvolvimento": "Desenvolvimento custom: orçamento sob consulta",
    }
```

```

    }

    for chave, descricao in base_dados_interna.items():
        if chave in servico_solicitado.lower():
            return descricao

    servicos_disponiveis = ",
".join(base_dados_interna.keys())
    return f"Nenhum serviço encontrado. Serviços
disponíveis: {servicos_disponiveis}"

@tool
def ler_ficheiro_texto(caminho_ficheiro: str) -> str:
    """Lê o conteúdo de um ficheiro de texto local.
    Use quando o utilizador pedir para analisar, resumir ou
    ler um ficheiro."""
    try:
        with open(caminho_ficheiro, 'r', encoding='utf-8')
as f:
            conteudo = f.read()
            return conteudo[:3000] # Limita para caber no
contexto
    except FileNotFoundError:
        return f"Ficheiro não encontrado:
{caminho_ficheiro}"
    except Exception as e:
        return f"Erro ao ler ficheiro: {str(e)}"

@tool
def calcular_iva(valor_sem_iva: str) -> str:
    """Calcula o valor com IVA a 23% dado um valor sem IVA.
    Use quando o utilizador precisar de calcular IVA sobre
    um valor."""
    try:
        valor = float(valor_sem_iva.replace(',', '.'))
        com_iva = valor * 1.23
        iva = valor * 0.23
        return f"Valor sem IVA: {valor:.2f}€ | IVA (23%):
{iva:.2f}€ | Total com IVA: {com_iva:.2f}€"
    except ValueError:
        return "Erro: forneça um número válido (ex: '1000'
ou '1000,50')"
```


A anatomia de uma ferramenta

Cada ferramenta segue sempre o mesmo padrão:

```
@tool                                     # ← 1. Regista
como ferramenta LangChain
def nome_da_ferramenta(argumento: str) -> str:
    """Descrição clara do que a ferramenta faz.
    O agente lê ESTA descrição para decidir quando usar a
    ferramenta.""" # ← 2. CRUCIAL

    # 3. Lógica da ferramenta
    resultado = fazer_algo(argumento)

    return str(resultado)                 # ← 4. Sempre
retornar string
```

:::warning A descrição (docstring) é o mecanismo de decisão O modelo de linguagem **não vê o código** da ferramenta. Apenas lê a descrição entre `"""`. Se a descrição for vaga ou incorrecta, o agente vai usar as ferramentas erradas — ou não as usar quando devia. :::

Criar ferramentas para o seu negócio

O poder do EmpresaIQ está em personalizá-lo para a sua empresa. Aqui ficam mais exemplos de ferramentas que pode criar:

mais_ferramentas.py (exemplos)

```
@tool
def verificar_stock(codigo_produto: str) -> str:
    """Verifica o stock disponível de um produto dado o seu
    código."""
    # Aqui ligaria à sua base de dados real
    return f"Produto {codigo_produto}: 42 unidades em stock"

@tool
```

```
def criar_nota_interna(conteudo: str) -> str:
    """Cria e guarda uma nota interna com o conteúdo
    fornecido."""
    import datetime
    timestamp =
datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    nome_ficheiro = f"nota_{timestamp}.txt"
    with open(nome_ficheiro, 'w', encoding='utf-8') as f:
        f.write(conteudo)
    return f"Nota guardada em: {nome_ficheiro}"

@tool
def consultar_nif(nif: str) -> str:
    """Valida e consulta informação de um NIF português."""
    # Validação básica
    if len(nif) == 9 and nif.isdigit():
        return f"NIF {nif} tem formato válido."
    return f"NIF {nif} inválido. Deve ter 9 dígitos."
```

:::tip Dica de produtividade Comece com poucas ferramentas (2-4) e vá adicionando à medida que percebe que padrões de uso são mais frequentes. Muitas ferramentas podem confundir o modelo em hardware limitado. :::

Verificar as ferramentas

Antes de avançar, confirme que o ficheiro não tem erros:

```
python -c "from tools import consultar_portfolio_empresaiq,
consultar_portal_base; print('Ferramentas OK!')"
```

Resumo

Neste capítulo:

- Percebemos o que são ferramentas e como o agente as usa
- Criámos o ficheiro `tools.py` com quatro ferramentas
- Aprendemos que a **descrição** (docstring) é o mecanismo que permite ao agente escolher a ferramenta certa

No próximo capítulo, juntamos tudo: modelo + ferramentas = agente EmpresaIQ completo.

Capítulo seguinte: *11. Construção do Agente* →

tools.py

```
import requests
from langchain.agents import tool

@tool
def consultar_portal_base(query_pesquisa: str) -> str:
    """Consulta contratos públicos no Portal Base do governo português."""

    url = f"https://base.gov.pt{query_pesquisa}"

    try:
        response = requests.get(url, timeout=5)

        if response.status_code == 200:
            return str(response.json()[2])

        return "Portal Base indisponível."

    except Exception as e:
        return f"Erro: {str(e)}"

@tool
def consultar_portfolio_empresaiq(servico_solicitado: str) -> str:
```

```
"""Consulta serviços e preços do portfolio interno da
EmpresaIQ."""

base_dados_interna = {
    "software": "EmpresaIQ Core: 5.000€/ano – Gestão
empresarial completa",
    "consultoria": "Consultoria IA: 120€/hora –
Implementação e formação",
    "cibersegurança": "EmpresaIQ Guard: 8.500€ –
Auditoria e protecção",
    "formacao": "Workshops IA: 800€/dia – Equipas até 20
pessoas",
    "desenvolvimento": "Desenvolvimento custom:
orçamento sob consulta",
}

for chave, descricao in base_dados_interna.items():
    if chave in servico_solicitado.lower():
        return descricao

return "Nenhum serviço encontrado. Serviços disponíveis:
" + ", ".join(base_dados_interna.keys())
```

Como Funcionam as Ferramentas

O decorador `@tool` do LangChain transforma uma função Python numa ferramenta que o agente pode invocar autonomamente.

```
Agente recebe pergunta
↓
Analisa quais ferramentas existem
↓
Decide qual ferramenta usar
↓
Chama a ferramenta com os argumentos certos
↓
Recebe resultado da ferramenta
↓
Formula resposta final
```

Anatomia de uma Ferramenta

```
@tool                                     # ← Regista como
ferramenta LangChain
def nome_da_ferramenta(argumento: str) -> str:
    """Descrição clara do que a ferramenta faz.""" # ← O
    agente lê isto!

    # Lógica da ferramenta
    resultado = fazer_algo(argumento)

    return str(resultado)                 # ← Sempre retornar
string
```

::important A docstring é fundamental O agente usa a **docstring** (texto entre `"""`) para decidir quando e como usar a ferramenta. Escreva descrições claras e precisas. :::

Adicionar Mais Ferramentas

Pode expandir facilmente com novas capacidades:

```
@tool
def ler_ficheiro_texto(caminho_ficheiro: str) -> str:
    """Lê o conteúdo de um ficheiro de texto local."""
    try:
        with open(caminho_ficheiro, 'r', encoding='utf-8')
as f:
            return f.read()[:2000] # Limita a 2000
caracteres
    except Exception as e:
        return f"Erro ao ler ficheiro: {str(e)}"

@tool
def calcular_iva(valor_sem_iva: str) -> str:
    """Calcula o valor com IVA a 23% dado um valor sem
IVA."""
    try:
        valor = float(valor_sem_iva.replace(',', '.'))
```

```
com_iva = valor * 1.23
return f"Valor sem IVA: {valor:.2f}€ | IVA (23%):
{valor * 0.23:.2f}€ | Total: {com_iva:.2f}€"
except Exception as e:
    return f"Erro: {str(e)}"
```

Boas Práticas

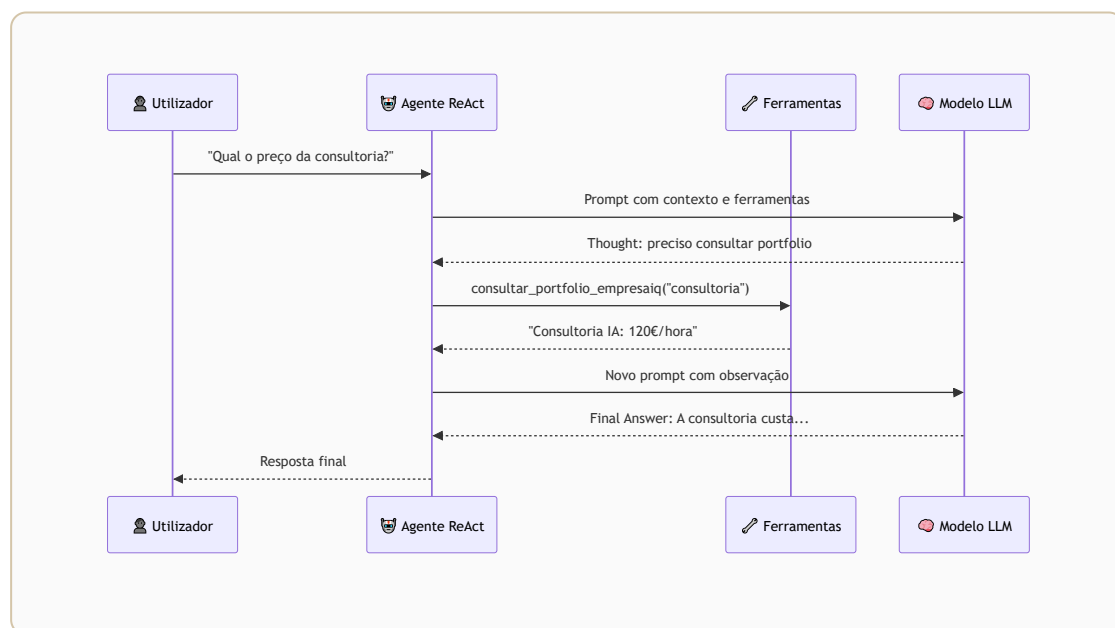
Prática	Porquê
Sempre retornar <code>str</code>	LangChain espera strings das ferramentas
Usar <code>try/except</code>	Erros de rede/disco não devem crashar o agente
Timeout nas chamadas HTTP	Evita o agente ficar bloqueado
Docstrings claras	O agente decide qual ferramenta usar com base nelas
Nomes descritivos	<code>consultar_portal_base</code> é melhor que <code>tool1</code>

Construção do Agente Empresarial

"Este é o capítulo central do livro. Aqui o EmpresaIQ ganha vida: modelo + ferramentas + raciocínio = agente completo."

O padrão ReAct — como o agente pensa

O EmpresaIQ usa o padrão **ReAct** (Reasoning + Acting). Em vez de responder directamente, o agente segue um ciclo de raciocínio que alterna entre pensar e agir:



Cada ciclo tem três partes:

- **Thought** — o agente raciocina sobre o que precisa
- **Action** — escolhe e chama uma ferramenta
- **Observation** — recebe o resultado e decide o próximo passo

Este processo repete-se até o agente ter informação suficiente para dar uma **Final Answer**.

O ficheiro principal — agente_local.py

Crie um ficheiro chamado `agente_local.py` na pasta `empresaiq-agent/`:

agente_local.py

```
from langchain_ollama import OllamaLLM
from langchain.agents import AgentExecutor,
create_react_agent
from langchain_core.prompts import PromptTemplate

from tools import (
    consultar_portal_base,
    consultar_portfolio_empresaiq,
    ler_ficheiro_texto,
    calcular_iva
)

# — 1. Ligar ao Modelo via Ollama


---



print("⌚ A ligar ao EmpresaIQ via Ollama...")

llm = OllamaLLM(
    model="empresaiq",                # Modelo
    personalizado_criado_no_Cap. 9
    base_url="http://localhost:11434", # Endereço padrão
    do Ollama
    temperature=0.1,                 # Baixa = mais
    preciso e determinístico
```



```

)

print("✅ EmpresaIQ pronto!")

# — 2. Definir Ferramentas

```

```

tools = [
    consultar_portal_base,
    consultar_portfolio_empresaiq,
    ler_ficheiro_texto,
    calcular_iva
]

# — 3. Prompt do Agente

```

```

template = """
Sés o Agente EmpresaIQ – um assistente empresarial
inteligente e profissional.
Respondes sempre em português de Portugal.
Usa as ferramentas disponíveis quando necessário para
responder com precisão.

Ferramentas disponíveis:
{tools}

Nomes das ferramentas: {tool_names}

Pergunta do utilizador: {input}

Thought: {agent_scratchpad}
"""

prompt = PromptTemplate.from_template(template)

# — 4. Criar o Agente

```

```

agent = create_react_agent(
    llm=llm,
    tools=tools,
    prompt=prompt
)

agent_executor = AgentExecutor(
    agent=agent,

```

```

tools=tools,
verbose=True,          # Mostra o raciocínio
Thought/Action
max_iterations=3,      # Máximo de ciclos por pergunta
handle_parsing_errors=True # Recupera de erros de
formato
)

# — 5. Interface Principal

```

```

if __name__ == "__main__":

    print("\n" + "="*50)
    print("  AGENTE EMPRESAIQ – IA LOCAL")
    print("  Modelo: empresaiq (Qwen2.5-3B) | Ollama")
    print("="*50)
    print("Escreva 'sair' para terminar.\n")

    pergunta = input("Pergunta: ")

    if pergunta.lower() != 'sair':
        resposta = agent_executor.invoke({"input":
pergunta})
        print("\n--- Resposta Final ---")
        print(resposta["output"])

```

Perceber cada parte do código

1. O OllamaLLM — ligar ao modelo

```

llm = OllamaLLM(
    model="empresaiq",          # Nome do modelo
    criado com Modelfile
    base_url="http://localhost:11434", # Ollama corre
    localmente
    temperature=0.1, # Próximo de 0 = mais previsível e
    preciso
)

```

Parâmetro	Valor Recomendado	O que controla
<code>model</code>	<code>"empresaiq"</code>	Nome do modelo no Ollama (ver <code>ollama list</code>)
<code>base_url</code>	<code>"http://localhost:11434"</code>	Endereço do servidor Ollama
<code>temperature</code>	0.1	Precisão vs criatividade (0=preciso, 1=criativo)
<code>max_iterations</code>	3	Número máximo de ciclos Thought/Action

2. O prompt — a identidade do agente

O **prompt** é o conjunto de instruções que define quem o agente é e como se comporta. Pode personalizá-lo para a sua empresa:

```
template = """
Sés o Agente EmpresaIQ – um assistente empresarial
inteligente e profissional.
Respondes sempre em português de Portugal.
# Adicione aqui instruções específicas da sua empresa:
# – Sector: jurídico / financeiro / saúde
# – Tom: formal / informal
# – Restrições: não discutes concorrência, etc.
...
"""
```

Executar o EmpresaIQ pela primeira vez

```
# Confirmar que o ambiente virtual está activo
venv\Scripts\activate      # Windows
source venv/bin/activate    # Linux/macOS

# Correr o agente
python agente_local.py
```

A primeira vez, vai ver:

```
⌚ A ligar ao EmpresaIQ via Ollama...
✅ EmpresaIQ pronto!

=====
  AGENTE EMPRESAIQ – IA LOCAL
  Modelo: empresaiq (Qwen2.5-3B) | Ollama
=====
Escreva 'sair' para terminar.

Pergunta: _
```

Ver o raciocínio em acção

Com `verbose=True` no `AgentExecutor`, consegue ver o processo ReAct completo:

```
Pergunta: "Qual o preço do software EmpresaIQ Core?"

> Entering new AgentExecutor chain...

Thought: Preciso de consultar o portfolio da EmpresaIQ para encontrar o preço do software.
Action: consultar_portfolio_empresaiq
Action Input: software
Observation: EmpresaIQ Core: 5.000€/ano – Gestão empresarial completa

Thought: Tenho a informação necessária.
```

Final Answer: O software EmpresaIQ Core tem um custo de 5.000€ por ano e inclui gestão empresarial completa.

Primeiros testes recomendados

Experimente estas perguntas para confirmar que o agente funciona correctamente:

- ✓ "Qual o preço do software EmpresaIQ Core?"
(deve usar `consultar_portfolio_empresaiq`)
- ✓ "Calcula o IVA sobre 2500 euros"
(deve usar `calcular_iva`)
- ✓ "Lê o ficheiro `requirements.txt`"
(deve usar `ler_ficheiro_texto`)
- ✓ "O que é quantização em inteligência artificial?"
(deve responder directamente, sem usar ferramentas)

:::tip Primeira execução mais lenta Na primeira execução após ligar o computador, o modelo demora mais a responder enquanto carrega para memória. As respostas seguintes são mais rápidas. :::

Resumo

Neste capítulo criou o coração do EmpresaIQ:

- O `agente_local.py` que junta modelo + ferramentas + raciocínio ReAct
- O agente responde a perguntas e usa ferramentas de forma autónoma
- A primeira versão do EmpresaIQ está funcional!

No próximo capítulo, vamos otimizar o desempenho para tirar o máximo partido do seu hardware.

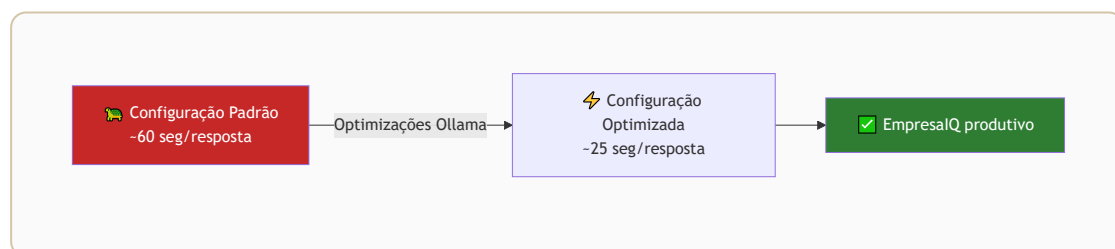
Capítulo seguinte: *12. Optimizações para CPU* →

Optimizações de Desempenho

"O EmpresaIQ já funciona. Agora vamos fazê-lo funcionar melhor — sem gastar mais um centimo em hardware."

Porque é que a optimização importa?

Num PC com 8 GB de RAM a correr um modelo em CPU, cada segundo conta. A diferença entre uma configuração optimizada e uma não optimizada pode ser de 2x a 3x na velocidade de resposta — sem mudar uma linha de código funcional, apenas ajustando parâmetros do Ollama.



Optimização 1 — Manter o modelo em memória (OLLAMA_KEEP_ALIVE)

Por padrão, o Ollama descarrega o modelo da RAM após 5 minutos de inactividade. Quando o agente é chamado de novo, precisa de carregar o modelo outra vez — o que demora 10–30 segundos.

Para respostas imediatas em produção, aumente o tempo de retenção:

```
# Windows (PowerShell) – definir antes de iniciar o Ollama
$env:OLLAMA_KEEP_ALIVE = "30m" # Manter 30 minutos em memória
ollama serve

# Linux / macOS
OLLAMA_KEEP_ALIVE=30m ollama serve
```

Ou defina permanentemente no sistema:

```
# Linux – adicionar ao /etc/environment ou ~/.profile
export OLLAMA_KEEP_ALIVE=30m

# Windows – nas Definições do Sistema > Variáveis de Ambiente
OLLAMA_KEEP_ALIVE = 30m
```

Valor	Quando usar
<code>5m</code> (padrão)	PC pessoal com pouca RAM
<code>30m</code>	Uso empresarial durante o dia
<code>60m</code>	Servidor sempre ligado
<code>-1</code>	Nunca descarregar (sempre em RAM)

Optimização 2 — Ajustar o contexto no Modelfile

O `num_ctx` (tamanho do contexto) define quantos tokens o modelo "lembra" de uma vez. Quanto maior, mais RAM consome. No Modelfile do EmpresaIQ (Cap. 9), configure conforme o hardware:

Modelfile

```
# Para 8 GB RAM — equilíbrio ideal
PARAMETER num_ctx 4096

# Para 4-6 GB RAM — mais rápido
PARAMETER num_ctx 2048

# Para 16 GB RAM — análise de documentos longos
PARAMETER num_ctx 8192
```

Após alterar o Modelfile, recrie o modelo:

```
ollama create empresaiq -f Modelfile
```

Optimização 3 — Temperature para agentes empresariais

A `temperature` controla a "criatividade" do modelo. Para um agente empresarial que precisa de responder com precisão:

Modelfile

```
PARAMETER temperature 0.05 # Máxima precisão — ideal para
dados e números
PARAMETER temperature 0.1 #  Recomendado — preciso mas
natural
PARAMETER temperature 0.3 # Ligeiramente mais criativo —
para texto narrativo
PARAMETER temperature 0.7 # Alta criatividade — para
brainstorming
```

Com `temperature` baixa, o agente:

- Segue instruções com mais rigor
- Usa ferramentas de forma mais consistente
- Produz menos "alucinações" (informação incorrecta inventada)

Optimização 4 — Libertar RAM antes de correr o agente

Antes de executar o EmpresaIQ, feche aplicações que consomem muita RAM:

- ❌ Fechar (maior impacto):
 - Browser com muitos separadores (Chrome/Edge pode usar 2-4 GB)
 - Docker Desktop
 - Adobe Creative Suite
 - Microsoft Teams / Slack (em segundo plano)
- ✅ Pode manter aberto:
 - Terminal / PowerShell
 - Editor de código (VS Code)
 - Explorador de ficheiros

Ver o que está a consumir mais RAM:

```
# Windows (PowerShell)
Get-Process | Sort-Object WorkingSet -Descending | `
  Select-Object -First 10 Name, @{N='RAM(MB)';E=
  {[math]::Round($_.WorkingSet/1MB,0)}}

# Linux
ps aux --sort=-%mem | head -10
```

Optimização 5 — Paralelismo para múltiplos utilizadores

Se o EmpresaIQ servir múltiplos utilizadores simultaneamente (ex: via API web), configure o Ollama para processar pedidos em paralelo:

```
# Permitir 2 pedidos simultâneos
$env:OLLAMA_NUM_PARALLEL = "2"

# Manter 2 modelos carregados em RAM (útil para múltiplos
modelos)
$env:OLLAMA_MAX_LOADED_MODELS = "2"
```

⚠ Para uso individual Para uso de uma única pessoa, mantenha `OLLAMA_NUM_PARALLEL=1`. O paralelismo divide os recursos de CPU entre pedidos e pode tornar cada resposta individual mais lenta. ⚠

Configuração final recomendada

Para um PC de escritório com 8 GB RAM usado por uma pessoa:

iniciar_empresaiq.ps1 (Windows)

```
# Script de arranque otimizado
$env:OLLAMA_KEEP_ALIVE = "30m"
$env:OLLAMA_NUM_PARALLEL = "1"
Start-Process ollama -ArgumentList "serve" -WindowStyle
Hidden
Write-Host "✅ Ollama iniciado – EmpresaIQ pronto!"
```

iniciar_empresaiq.sh (Linux/macOS)

```
#!/bin/bash
export OLLAMA_KEEP_ALIVE=30m
export OLLAMA_NUM_PARALLEL=1
ollama serve &
echo "✅ Ollama iniciado – EmpresaIQ pronto!"
```

E o Modelfile otimizado:

Modelfile (configuração otimizada)

```
FROM qwen2.5:3b
```

```
SYSTEM ""
```

```
És o EmpresaIQ...
```

```
""
```

```
PARAMETER temperature 0.1
```

```
PARAMETER top_p 0.9
```

```
PARAMETER num_ctx 4096          # Ajuste conforme RAM  
disponível
```

```
PARAMETER repeat_penalty 1.1
```

Medir a velocidade do EmpresaIQ

Se quiser medir a velocidade de resposta antes e depois das otimizações:

benchmark_simples.py

```
import time
import ollama

start = time.time()
resposta = ollama.chat(
    model='empresaiq',
    messages=[{'role': 'user', 'content': 'Responde apenas:  
olá'}]
)
elapsed = time.time() - start

tokens = len(resposta['message']['content'].split())
print(f"Tempo de resposta: {elapsed:.1f} segundos")
print(f"Velocidade: ~{tokens/elapsed:.0f} palavras/segundo")
```

Resumo

As 5 otimizações por ordem de impacto:

Optimização	Impacto Esperado	Dificuldade
1. Acertar <code>n_threads</code>	★★★★★ muito alto	Trivial
2. Ajustar <code>n_ctx</code>	★★★ alto	Trivial
3. Recompilar com AVX2	★★★★ alto	Fácil
4. Libertar RAM	★★★ médio	Manual
5. <code>temperature</code> baixa	★★ qualidade	Trivial

Com estas configurações, o EmpresaIQ já é um agente produtivo em qualquer PC de escritório.

Capítulo seguinte: [13. Interface de Chat Local →](#)

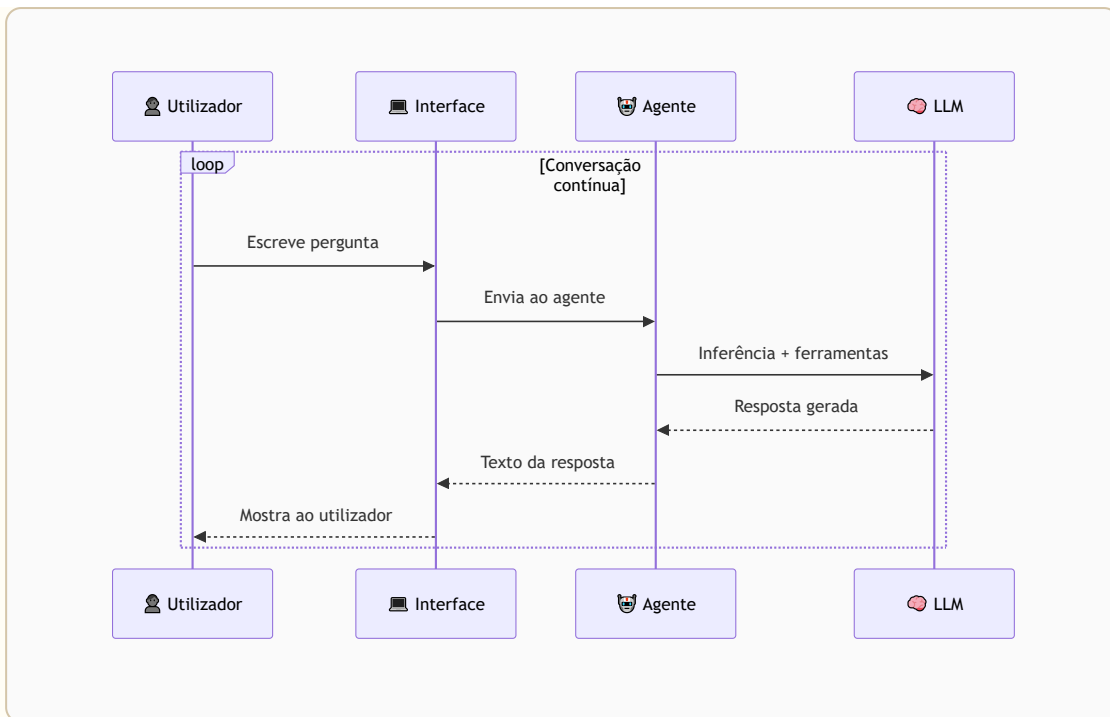
Interface de Chat Local

"Um agente que só responde a uma pergunta de cada vez e depois termina é como um funcionário que sai do escritório depois de cada e-mail. Neste capítulo, damos ao EmpresaIQ uma interface de conversação contínua."

O que vamos construir

Até aqui, o agente responde a uma pergunta e termina. Neste capítulo, vamos criar:

1. **Chat em loop no terminal** — a forma mais simples de conversar continuamente
2. **Chat com histórico de sessão** — o agente lembra-se do contexto da conversa
3. **Interface web com Flask** — uma página no browser para conversar com o agente



Opção 1 — Chat em loop no terminal

A forma mais simples: substitua o bloco `if __name__ == "__main__":` no `agente_local.py` por este loop:

`agente_local.py` (substitua o bloco main)

```
if __name__ == "__main__":  
  
    print("\n" + "="*50)  
    print("  AGENTE EMPRESAIQ – IA LOCAL")  
    print("  Modelo: empresaiq (Qwen2.5-3B) | Ollama")  
    print("="*50)  
    print("Escreva 'sair' para terminar.\n")  
  
    while True:  
        try:  
            pergunta = input("Você: ").strip()  
  
            if not pergunta:  
                continue  
  
            if pergunta.lower() in ("sair", "exit", "quit"):  
                print("Até logo!")
```

```

        break

    print("Agente a processar...")

    resposta = agent_executor.invoke({"input":
pergunta})
    print(f"\nAgente: {resposta['output']}\n")
    print("-" * 40)

except KeyboardInterrupt:
    print("\nInterrompido. Até logo!")
    break
except Exception as e:
    print(f"Erro: {e}")
    continue

```

Agora o agente fica activo até escrever .

Opção 2 — Chat com histórico de sessão

O agente base não se lembra das mensagens anteriores — cada pergunta é tratada de forma independente. Para adicionar memória de curto prazo:

agente_com_memoria.py

```

from langchain_ollama import OllamaLLM
from langchain.agents import AgentExecutor,
create_react_agent
from langchain_core.prompts import PromptTemplate
from langchain.memory import ConversationBufferWindowMemory

from tools import consultar_portfolio_empresaiq,
consultar_portal_base, calcular_iva

# Ligar ao modelo via Ollama
llm = OllamaLLM(
    model="empresaiq",
    base_url="http://localhost:11434",
    temperature=0.1

```



```

)

# Memória das últimas 3 trocas (mais aumenta o consumo de contexto)
memory = ConversationBufferWindowMemory(
    k=3,
    memory_key="chat_history",
    return_messages=False # False = texto, True = objectos
)

tools = [consultar_portfolio_empresaiq,
consultar_portal_base, calcular_iva]

# Prompt actualizado com histórico
template = """
Sés o Agente EmpresaIQ – assistente empresarial em português de Portugal.

Histórico da conversa:
{chat_history}

Ferramentas disponíveis: {tools}
Nomes: {tool_names}

Pergunta actual: {input}
Thought: {agent_scratchpad}
"""

agent = create_react_agent(llm, tools,
PromptTemplate.from_template(template))

executor = AgentExecutor(
    agent=agent, tools=tools, memory=memory,
    verbose=False, max_iterations=3,
    handle_parsing_errors=True
)

# Loop de chat
print("EmpresaIQ com memória (lembra as últimas 3 trocas). Escreva 'sair' para terminar.\n")

while True:
    pergunta = input("Você: ").strip()
    if pergunta.lower() in ("sair", "exit"):
        break
    if not pergunta:
        continue

```

```
resposta = executor.invoke({"input": pergunta})
print(f"\nAgente: {resposta['output']}\n" + "-"*40)
```

:::caution Memória e RAM em 8 GB Cada mensagem guardada ocupa tokens de contexto. Com `k=3` (3 trocas = 6 mensagens), já pode consumir 500–800 tokens extra. Se o agente ficar mais lento, reduza para `k=2`. :::

Opção 3 — Interface web com Flask

Para uma interface mais agradável, acessível pelo browser:

```
# Instalar Flask
pip install flask
```

web_chat.py

```
from flask import Flask, request, jsonify,
render_template_string
from agente_local import agent_executor

app = Flask(__name__)

HTML = """
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <title>EmpresaIQ – Agente Local</title>
    <style>
        body { font-family: Segoe UI, Arial, sans-serif; max-
width: 750px; margin: 40px auto; padding: 20px; background:
#f5f5f5; }
        h2 { color: #1D2951; }
        #chat { background: white; border: 1px solid #ddd;
border-radius: 8px; height: 400px; overflow-y: auto; padding:
15px; margin-bottom: 10px; }
        .user { color: #0078d4; margin: 8px 0; }
```



```

@app.route('/')
def index():
    return render_template_string(HTML)

@app.route('/chat', methods=['POST'])
def chat():
    dados = request.get_json()
    pergunta = dados.get('pergunta', '')
    if not pergunta:
        return jsonify({"resposta": "Pergunta vazia."})
    resultado = agent_executor.invoke({"input": pergunta})
    return jsonify({"resposta": resultado["output"]})

if __name__ == '__main__':
    print("Interface web disponível em:
    http://localhost:5000")
    app.run(host='127.0.0.1', port=5000, debug=False)

```

Inicie com:

```
python web_chat.py
```

Abra o browser em **http://localhost:5000** e comece a conversar.

:::info Apenas para uso local O servidor Flask está configurado em `127.0.0.1` — só acessível no próprio computador. Para partilhar na rede interna da empresa, use `host='0.0.0.0'` e adicione autenticação antes de o fazer. :::

Comparar as três opções

Opção	Facilidade	Aspecto	Memória	Reco
Loop no terminal	★ ★ ★ ★ ★	Terminal	Sem histórico	Teste
Terminal com memória	★ ★ ★ ★	Terminal	✅ Histórico curto	Uso d pesso
Interface web Flask	★ ★ ★	Browser	Configurável	Demo e equi

Resumo

Neste capítulo:

- Criou um loop de conversação contínua no terminal
- Adicionou histórico de sessão com `ConversationBufferWindowMemory`
- Construiu uma interface web simples com Flask

O EmpresaIQ está agora conversável de várias formas. No próximo capítulo, vamos fazê-lo trabalhar **automaticamente**, sem necessitar de intervenção humana.

Capítulo seguinte: 14. Automação por Hora →

interface de chat — adicionar ao agente_local.py

```
if __name__ == "__main__":

    print("\n" + "="*50)
    print("  AGENTE EMPRESAIQ – IA LOCAL")
    print("  Modelo: empresaiq (Qwen2.5-3B) | Ollama")
    print("="*50)
    print("Escreva 'sair' para terminar.\n")

    while True:
        try:
            pergunta = input("Você: ").strip()

            if not pergunta:
                continue

            if pergunta.lower() in ("sair", "exit", "quit"):
                print("Até logo!")
                break

            print("Agente a processar...")

            resposta = agent_executor.invoke({
                "input": pergunta
            })

            print(f"\nAgente: {resposta['output']}\n")
            print("-" * 40)

        except KeyboardInterrupt:
            print("\nInterrompido pelo utilizador.")
            break
        except Exception as e:
            print(f"Erro: {e}")
            continue
```

Chat com Histórico de Conversação

Para o agente "lembrar" as mensagens anteriores da sessão:

chat com memória de sessão

```
from langchain.memory import ConversationBufferWindowMemory

# Memória das últimas 5 trocas
memory = ConversationBufferWindowMemory(
    k=5,
    memory_key="chat_history",
    return_messages=True
)

# Prompt actualizado com histórico
template_com_memoria = """
És o Agente EmpresaIQ.

Histórico da conversa:
{chat_history}

Ferramentas: {tools}
Nomes: {tool_names}

Pergunta actual: {input}
Thought: {agent_scratchpad}
"""

prompt_memoria =
PromptTemplate.from_template(template_com_memoria)

agent_com_memoria = create_react_agent(llm, tools,
prompt_memoria)

executor_com_memoria = AgentExecutor(
    agent=agent_com_memoria,
    tools=tools,
    memory=memory,
    verbose=False,
    max_iterations=3,
    handle_parsing_errors=True
)
```

⚠ Memória e RAM Cada mensagem guardada na memória ocupa tokens de contexto. Com `k=5` (5 trocas), o contexto pode crescer rapidamente. Em hardware limitado, use `k=3` ou `k=2`. ⚠

Interface Web Simples com Flask

Para uma interface web básica:

```
pip install flask
```

web_chat.py

```
from flask import Flask, request, jsonify, render_template_string

app = Flask(__name__)

HTML = """
<!DOCTYPE html>
<html>
<head>
    <title>EmpresaIQ Agent</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 700px; margin: 40px auto; padding: 20px; }
        #chat { border: 1px solid #ddd; height: 400px; overflow-y: auto; padding: 15px; margin-bottom: 10px; }
        input { width: 80%; padding: 8px; }
        button { padding: 8px 16px; background: #0078d4; color: white; border: none; cursor: pointer; }
        .user { color: #0078d4; }
        .agent { color: #107c10; }
    </style>
</head>
<body>
    <h2>🤖 Agente EmpresaIQ</h2>
    <div id="chat"></div>
    <input id="input" type="text" placeholder="Escreva a sua pergunta...">
    <button onclick="enviar()">Enviar</button>
</body>
</html>
"""

@app.route("/")
def index():
    return render_template_string(HTML)
```



```

<script>
    async function enviar() {
        const input = document.getElementById('input');
        const chat = document.getElementById('chat');
        const pergunta = input.value.trim();
        if (!pergunta) return;
        chat.innerHTML += `<p class="user"><b>Você:</b>
${pergunta}</p>`;
        input.value = '';
        const r = await fetch('/chat', {method:'POST',
headers:{'Content-Type':'application/json'}, body:
JSON.stringify({pergunta})});
        const data = await r.json();
        chat.innerHTML += `<p class="agent"><b>Agente:
</b> ${data.resposta}</p>`;
        chat.scrollTop = chat.scrollHeight;
    }

document.getElementById('input').addEventListener('keypress',
e => { if(e.key==='Enter') enviar(); });
</script>
</body>
</html>
"""

@app.route('/')
def index():
    return render_template_string(HTML)

@app.route('/chat', methods=['POST'])
def chat():
    data = request.get_json()
    pergunta = data.get('pergunta', '')
    resposta = agent_executor.invoke({"input": pergunta})
    return jsonify({"resposta": resposta["output"]})

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=5000, debug=False)

```

Aceda em: **http://localhost:5000**

:::info Segurança O servidor Flask está configurado apenas para
127.0.0.1 (localhost). Nunca exponha o agente diretamente à
internet sem autenticação. :::

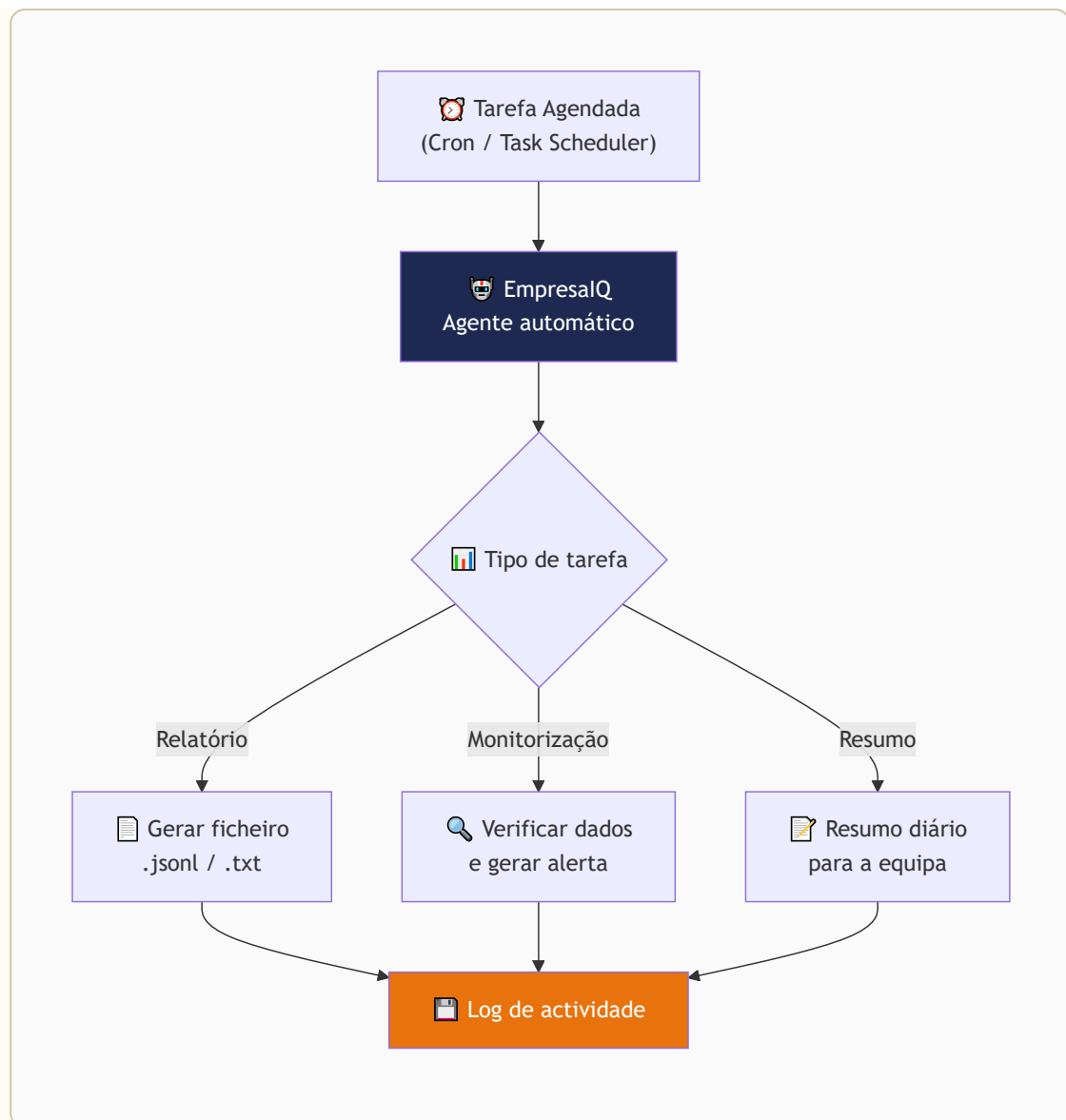
Automatização por Hora

"Um agente que precisa de um humano para cada tarefa ainda é uma ferramenta. Um agente que trabalha sozinho é um colaborador."

O que vamos construir

Neste capítulo, o EmpresaIQ aprende a trabalhar sem que ninguém o inicie manualmente:

- De hora a hora, gera relatórios e guarda-os em ficheiro
- Todas as manhãs, prepara um resumo para a equipa
- Monitoriza dados e gera alertas automaticamente



Passo 1 — Criar o script automático

Primeiro, crie um script que o agente possa executar sem interação humana:

agente_automatico.py

```
import sys
from agente_local import agent_executor
from datetime import datetime
import json
```

```

def executar_tarefa(pergunta: str, ficheiro_output: str =
"log_agente.jsonl"):
    """Executa uma tarefa no agente e guarda o resultado."""

    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M")
    print(f"[{timestamp}] A processar: {pergunta}")

    try:
        resultado = agent_executor.invoke({"input":
pergunta})
        resposta = resultado["output"]
    except Exception as e:
        resposta = f"ERRO: {str(e)}"

    print(f"Resposta: {resposta}")

    # Guardar em ficheiro JSON Lines (uma linha por registo)
    entrada = {
        "timestamp": datetime.now().isoformat(),
        "pergunta": pergunta,
        "resposta": resposta
    }

    with open(ficheiro_output, 'a', encoding='utf-8') as f:
        f.write(json.dumps(entrada, ensure_ascii=False) +
"\n")

    return resposta

if __name__ == "__main__":
    # Tarefa padrão ou argumento da linha de comandos
    tarefa = sys.argv[1] if len(sys.argv) > 1 else "Quais os
serviços disponíveis da EmpresaIQ e os seus preços?"
    executar_tarefa(tarefa)

```

Testar manualmente:

```

python agente_automatico.py "Gera um resumo dos serviços
EmpresaIQ"
# O resultado é guardado em log_agente.jsonl

```

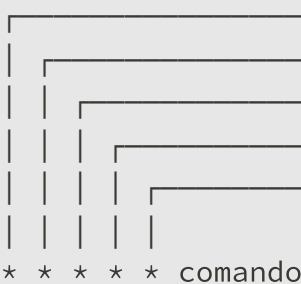
Passo 2 — Agendar no Linux (Cron)

```
# Abrir o editor de cron  
crontab -e
```

Adicione as linhas necessárias:

```
# Correr de hora a hora  
0 * * * * cd /home/user/empresaiq-agent && source  
venv/bin/activate && python agente_automatico.py >>  
/var/log/agente.log 2>&1  
  
# Correr todos os dias às 08:00  
0 8 * * * cd /home/user/empresaiq-agent && source  
venv/bin/activate && python agente_automatico.py "Gera  
relatório diário de serviços" >> /var/log/agente.log 2>&1  
  
# Correr às 2ª feira às 09:00  
0 9 * * 1 cd /home/user/empresaiq-agent && source  
venv/bin/activate && python agente_automatico.py "Resumo  
semanal do portfolio" >> /var/log/agente.log 2>&1
```

Sintaxe do Cron:



O diagrama ilustra a estrutura da linha de comando do Cron. À esquerda, há uma sequência de cinco caracteres de controle de tempo, cada um precedido por uma barra vertical: '|', '|', '|', '|', '|'. À direita, há uma sequência de caracteres de controle de tempo: '|', '|', '|', '|', '|'. Linhas horizontais conectam os caracteres de controle de tempo à esquerda aos seus respectivos campos de tempo à direita. As linhas são: a primeira linha conecta o primeiro '|' à 'minuto (0-59)', a segunda linha conecta o segundo '|' à 'hora (0-23)', a terceira linha conecta o terceiro '|' à 'dia do mês (1-31)', a quarta linha conecta o quarto '|' à 'mês (1-12)', e a quinta linha conecta o quinto '|' à 'dia da semana (0-7)'. Abaixo da sequência de caracteres de controle de tempo, há a palavra 'comando' precedida por um espaço.

```
* * * * * comando
```

Passo 2 (alternativa) — Agendar no Windows

Via PowerShell

```
# Criar tarefa para correr de hora a hora  
$Acao = New-ScheduledTaskAction `
```

```
-Execute "$env:USERPROFILE\empresaiq-agent\venv\Scripts\python.exe" `
-Argument "agente_automatico.py" `
-WorkingDirectory "$env:USERPROFILE\empresaiq-agent"

$Gatilho = New-ScheduledTaskTrigger `
-RepetitionInterval (New-TimeSpan -Hours 1) `
-Once -At (Get-Date)

Register-ScheduledTask `
-TaskName "EmpresaIQ-Automatico" `
-Action $Acao `
-Trigger $Gatilho `
-Description "EmpresaIQ - agente automático"
```

Via interface gráfica

1. Abra o **Agendador de Tarefas** (`taskschd.msc`)
 2. Clique **Criar Tarefa Básica...**
 3. Nome: `EmpresaIQ Automático`
 4. Trigger: **Diariamente às 08:00**
 5. Ação: **Iniciar um programa**
 - o Programa: `C:\empresaiq-agent\venv\Scripts\python.exe`
 - o Argumentos: `agente_automatico.py "Relatório diário"`
 - o Iniciar em: `C:\empresaiq-agent`
-

Casos de uso reais

Tarefa	Agendamento	Ficheiro de saída
Relatório diário de portfolio	Todos os dias 08:00	relatorio_diario.jsonl
Verificação de contratos públicos	De 2 em 2 horas	contratos_alerta.jsonl
Resumo semanal	2ª feira 09:00	resumo_semanal.jsonl
Cálculo de IVA em lote	Mensalmente	iva_mensal.jsonl

:::tip Ver os resultados Leia o ficheiro `log_agente.jsonl` para ver todos os resultados:

```
import json
with open('log_agente.jsonl') as f:
    for linha in f:
        entrada = json.loads(linha)
        print(f"[{entrada['timestamp']}]
{entrada['resposta'][:100]}")
```

:::

Resumo

Neste capítulo:

- Criou o `agente_automatico.py` que corre sem intervenção humana
- Aprendeu a agendar tarefas com Cron (Linux) e Agendador de Tarefas (Windows)
- Definiu casos de uso reais para automatização empresarial

No próximo capítulo, abordamos um tema crítico para qualquer empresa: segurança e privacidade dos dados tratados pelo EmpresaIQ.

Capítulo seguinte: *15. Segurança e Privacidade* →

Criar um Script Automatizável

Primeiro, transforme o agente num script não-interactivo:

`agente_automatico.py`

```
import sys
from agente_local import agent_executor
from datetime import datetime
import json

def executar_tarefa(pergunta: str, ficheiro_output: str = None):
    """Executa uma tarefa no agente e guarda o resultado."""

    print(f"[{datetime.now().strftime('%Y-%m-%d %H:%M')}] A processar: {pergunta}")

    resultado = agent_executor.invoke({"input": pergunta})
    resposta = resultado["output"]

    print(f"Resposta: {resposta}")

    if ficheiro_output:
```

```

        with open(ficheiro_output, 'a', encoding='utf-8') as
f:
            entrada = {
                "timestamp": datetime.now().isoformat(),
                "pergunta": pergunta,
                "resposta": resposta
            }
            f.write(json.dumps(entrada, ensure_ascii=False)
+ "\n")

        return resposta

if __name__ == "__main__":
    # Tarefa padrão ou argumento da linha de comandos
    tarefa = sys.argv[1] if len(sys.argv) > 1 else "Resume
os serviços disponíveis da EmpresaIQ"
    executar_tarefa(tarefa,
ficheiro_output="log_agente.jsonl")

```

Linux — Cron

```

# Abrir editor de cron
crontab -e

```

Exemplos de agendamento

```

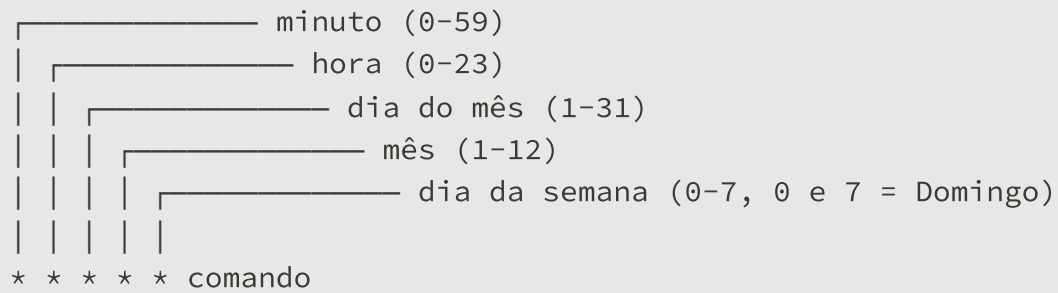
# Correr a cada hora
0 * * * * cd /home/user/empresaiq-agent && source
venv/bin/activate && python agente_automatico.py >>
/var/log/agente.log 2>&1

# Correr todos os dias às 08:00
0 8 * * * cd /home/user/empresaiq-agent && source
venv/bin/activate && python agente_automatico.py "Gera
relatório diário de serviços" >> /var/log/agente.log 2>&1

# Correr às 2ª feira às 09:00
0 9 * * 1 cd /home/user/empresaiq-agent && source
venv/bin/activate && python agente_automatico.py "Resumo
semanal do portfolio" >> /var/log/agente.log 2>&1

```

Sintaxe Cron



Windows — Agendador de Tarefas

Via PowerShell (linha de comandos)

```
# Criar tarefa agendada para correr de hora a hora
$action = New-ScheduledTaskAction `
    -Execute "python" `
    -Argument "C:\empresaiq-agent\agente_automatico.py" `
    -WorkingDirectory "C:\empresaiq-agent"

$trigger = New-ScheduledTaskTrigger -RepetitionInterval
(New-TimeSpan -Hours 1) -Once -At (Get-Date)

Register-ScheduledTask `
    -TaskName "EmpresaIQ-Agente-Automatico" `
    -Action $action `
    -Trigger $trigger `
    -Description "Agente IA EmpresaIQ – execução automática"
```

Via Interface Gráfica

1. Abra **Agendador de Tarefas** (Task Scheduler)
2. Clique em **Criar Tarefa Básica...**
3. Nome:
4. Trigger: **Diariamente ou Quando o computador iniciar**
5. Acção: **Iniciar um programa**
 - o Programa:

- Argumentos: `agente_automatico.py "Tarefa a executar"`
- Iniciar em: `C:\empresaiq-agent`

Casos de Uso Automáticos

Tarefa	Agendamento	Benefício
Relatório diário de portfolio	Todos os dias 08:00	Preparação para reuniões
Verificação de contratos públicos	De 2 em 2 horas	Monitorização contínua
Resumo semanal	Segunda às 09:00	Visão geral da semana
Alerta de novos concursos	Cada 30 minutos	Oportunidades em tempo real

DICA

Guarde os resultados em ficheiros `.jsonl` (JSON Lines) — são fáceis de analisar com Python ou importar para Excel/Power BI.

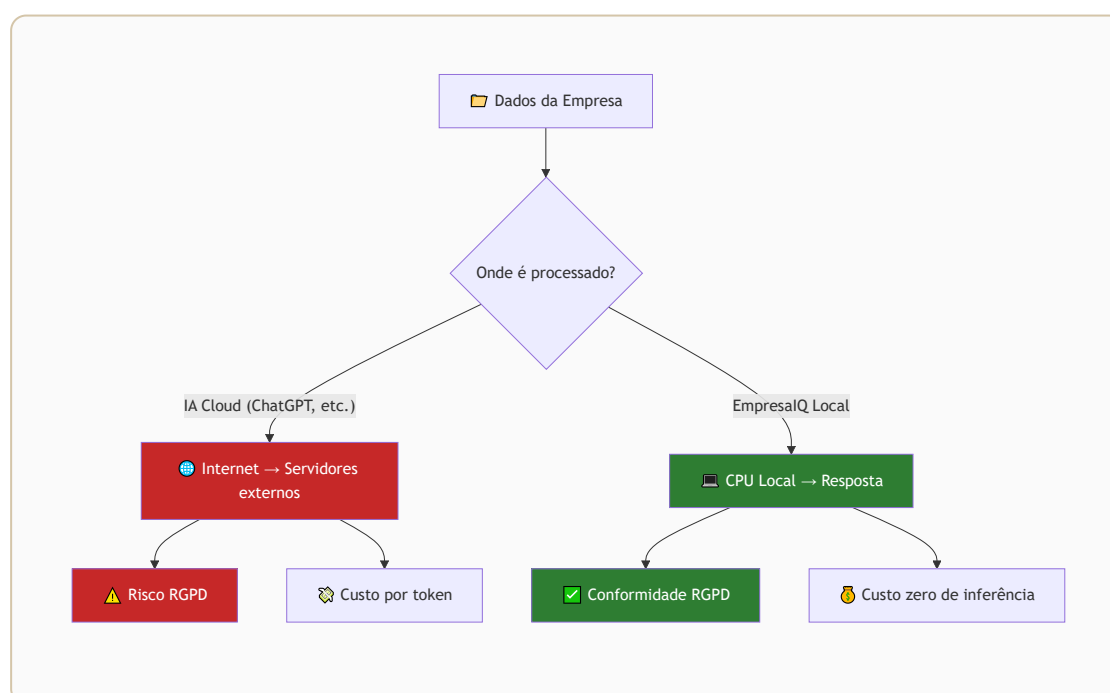
Segurança e Privacidade

"Com IA local, a privacidade não é uma promessa nem uma política. É uma garantia arquitetural: os dados nunca saem do seu computador."

A vantagem fundamental da IA local

Quando usa uma API de IA na cloud (ChatGPT, Gemini, Claude), cada pergunta que faz — incluindo qualquer dado de clientes ou informação confidencial que cole no prompt — **viaja pela internet e é processada em servidores externos**.

Com o EmpresaIQ local, isto não acontece:



Conformidade RGPD

O Regulamento Geral de Proteção de Dados (RGPD) exige várias garantias. Veja como o EmpresaIQ se posiciona:

Requisito RGPD	EmpresaIQ Local	APIs Cloud
Dados não saem da UE	✓ Sempre	⚠ Depende do contrato
Controlo total dos dados	✓ Total	✗ Limitado
Direito ao esquecimento	✓ Delete local simples	⚠ Processo complexo
Minimização de dados	✓ Só usa o que precisa	⚠ Logs nos servidores deles
Avaliação de impacto (DPIA)	✓ Simples	✗ Muito complexa
Auditoria de acessos	✓ Controlo total	✗ Visível apenas parcialmente

Sectores que beneficiam mais

Advocacia e Notariado

O sigilo profissional é obrigatório. Contratos, testamentos e dados de clientes são protegidos por lei. Com o EmpresaIQ:

- Análise de contratos sem expor conteúdo a terceiros
- Redação de pareceres com IA confidencial
- Pesquisa jurídica em documentos internos

Contabilidade e Finanças

Dados fiscais e financeiros são altamente sensíveis. O sigilo bancário e fiscal não admite excepções:

- Análise de balanços sem enviar números para servidores externos
- Detecção de irregularidades em dados internos
- Relatórios financeiros gerados localmente

Saúde

Dados de saúde são **categoria especial** no RGPD — obrigam a protecção reforçada e as multas por violação podem ser enormes:

- Apoio a triagem e documentação clínica
- Análise de relatórios médicos internos
- Sem partilha de dados com entidades externas

Sector Público

Dados classificados e obrigação de soberania digital impedem o uso de cloud estrangeira para dados sensíveis:

- Processamento de documentos internos classificados
 - Automatização de processos com auditabilidade total
-

Boas práticas de segurança

1. Usar sempre ambiente virtual isolado

```
# Isola as dependências do projecto do resto do sistema
python -m venv venv
```

Nunca instale as dependências directamente no Python do sistema.

2. Nunca expor o agente sem autenticação

```
# CORRECTO – apenas localhost
app.run(host='127.0.0.1', port=5000)

# ERRADO – expõe toda a rede sem protecção
app.run(host='0.0.0.0', port=5000) # Nunca sem
autenticação!
```

Se precisar de expor o agente na rede interna, adicione primeiro autenticação básica:

```
from flask import Flask, request, jsonify
from functools import wraps

# Token simples – num ambiente real use JWT ou OAuth
TOKEN_VALIDO = "empresaiq-token-secreto-aqui"

def requer_token(f):
    @wraps(f)
    def decorada(*args, **kwargs):
        token = request.headers.get('Authorization')
        if token != f"Bearer {TOKEN_VALIDO}":
            return jsonify({"erro": "Acesso negado"}), 401
        return f(*args, **kwargs)
    return decorada

@app.route('/chat', methods=['POST'])
@requer_token
def chat():
    ...
```


3. Limpar logs antigos automaticamente

Não guarde logs indefinidamente. Aplique uma política de retenção:

limpar_logs.py

```
import os
import glob
from datetime import datetime, timedelta

def limpar_logs_antigos(pasta=".", dias=30):
    """Remove ficheiros de log com mais de X dias."""
    limite = datetime.now() - timedelta(days=dias)
    removidos = 0
    for ficheiro in glob.glob(f"{pasta}/*.jsonl") + glob.glob(f"{pasta}/*.log"):
        if os.path.getmtime(ficheiro) < limite.timestamp():
            os.remove(ficheiro)
            removidos += 1
    print(f"{removidos} ficheiros de log removidos.")

# Executar mensalmente via cron:
# 0 3 1 * * python /path/to/limpar_logs.py
```

4. Encriptar logs com dados sensíveis

Se os logs contiverem dados de clientes, encripte-os:

```
pip install cryptography
```

encriptar_log.py

```
from cryptography.fernet import Fernet

# AVISO: Guarde a chave em local seguro (não no mesmo
# ficheiro nem no git!)
chave = Fernet.generate_key() # Gera uma vez e guarda
cifra = Fernet(chave)

def encriptar_ficheiro(caminho: str):
    with open(caminho, 'rb') as f:
        dados = f.read()
    with open(caminho + '.enc', 'wb') as f:
```

```
f.write(cifra.encrypt(dados))
os.remove(caminho) # Remove o original
print(f"Ficheiro encriptado: {caminho}.enc")
```

:::danger Não inclua a chave no repositório A chave de encriptação nunca deve estar no código nem no git. Use variáveis de ambiente ou um gestor de segredos (Azure Key Vault, HashiCorp Vault). :::

Checklist de segurança

Antes de usar o EmpresaIQ com dados reais de clientes, confirme:

- ☐ Ambiente virtual isolado criado e activo
 - ☐ Ficheiro `.gitignore` inclui `*.log`, `*.jsonl`, `*.enc`, `chave_*`
 - ☐ Interface web só acessível em `127.0.0.1` (ou com autenticação)
 - ☐ Política de retenção de logs definida (máx. 30 dias recomendado)
 - ☐ Logs com dados sensíveis encriptados
 - ☐ Ninguém com acesso não autorizado ao servidor
-

Resumo

Neste capítulo:

- Percebemos porque a IA local oferece garantias de privacidade superiores à cloud
- Viemos como o EmpresaIQ se enquadra no RGPD
- Aplicámos quatro boas práticas de segurança essenciais

Nas próximas versões do EmpresaIQ, há muito mais para explorar. O próximo capítulo apresenta o roteiro de melhorias futuras.

Capítulo seguinte: *16. Melhorias Futuras* →

A Vantagem Fundamental

Com IA local, a segurança está garantida por arquitectura — não por política ou contrato.

Cloud AI:

Dados → Internet → Servidores externos → Resposta

⚠️ Dados viajam fora do seu controlo

IA Local EmpresaIQ:

Dados → CPU local → Resposta

✅ Dados nunca saem do seu computador

Conformidade RGPD

O Regulamento Geral de Protecção de Dados (RGPD) exige que:

Requisito RGPD	IA Local	APIs Cloud
Dados não saem da UE	✅ Sempre	⚠️ Depende do contrato
Controlo total dos dados	✅ Sim	❌ Limitado
Direito ao esquecimento	✅ Delete local	⚠️ Complexo
Minimização de dados	✅ Controlo total	⚠️ Logs nos servidores
Avaliação de impacto (DPIA)	✅ Simples	❌ Complexa

Sectores que Beneficiam Mais

Advocacia / Notariado

Porquê IA local é essencial:

- Sigilo profissional obrigatório
- Contratos e testamentos são confidenciais
- Dados de clientes protegidos por lei

Uso prático:

- Análise de contratos internamente
- Redacção de pareceres
- Pesquisa jurídica

Contabilidade / Finanças

Porquê IA local é essencial:

- Dados fiscais e financeiros altamente sensíveis
- Sigilo bancário e fiscal
- Risco de compliance se dados saírem para fora

Uso prático:

- Análise de balanços
- Detecção de irregularidades
- Relatórios financeiros automáticos

Saúde

Porquê IA local é essencial:

- Dados de saúde são categoria especial no RGPD
- Obrigação legal de confidencialidade
- Multas enormes por violação

Uso prático:

- Triagem de sintomas
- Análise de relatórios médicos
- Apoio a diagnóstico

Sector Público

Porquê IA local é essencial:

- Dados classificados e sensíveis
- Obrigação de soberania digital

- Impossibilidade de usar cloud estrangeira para dados sensíveis

Uso prático:

- Análise de candidaturas
- Processamento de documentos internos
- Automatização de processos

Boas Práticas de Segurança

1. Isolar o Ambiente

```
# Usar sempre ambiente virtual isolado
python -m venv venv
```

2. Não Expor o Agente à Rede

```
# CORRECTO – apenas localhost
app.run(host='127.0.0.1', port=5000)

# ERRADO – expõe a toda a rede
app.run(host='0.0.0.0', port=5000) # ❌ Nunca sem
autenticação
```

3. Limpar Logs Automaticamente

```
import os
import glob
from datetime import datetime, timedelta

def limpar_logs_antigos(pasta=".", dias=30):
    """Remove logs com mais de X dias."""
    limite = datetime.now() - timedelta(days=dias)
    for ficheiro in glob.glob(f"{pasta}/*.log"):
        if os.path.getmtime(ficheiro) < limite.timestamp():
            os.remove(ficheiro)
```

4. Encriptar Logs Sensíveis

```
# Instalar
pip install cryptography

# Exemplo simples de encriptação de ficheiro de log
```

```
from cryptography.fernet import Fernet

# Gerar chave (guardar em local seguro!)
chave = Fernet.generate_key()
cifra = Fernet(chave)

# Encriptar
with open('log_agente.jsonl', 'rb') as f:
    dados = f.read()
dados_cifrados = cifra.encrypt(dados)

with open('log_agente.enc', 'wb') as f:
    f.write(dados_cifrados)
```

Auditoria de Acessos

Registe sempre quem perguntou o quê:

```
import logging

logging.basicConfig(
    filename='auditoria.log',
    level=logging.INFO,
    format='%(asctime)s - %(message)s'
)

def registrar_acesso(utilizador: str, pergunta: str):
    logging.info(f"Utilizador: {utilizador} | Pergunta: {pergunta[:100]}")
```

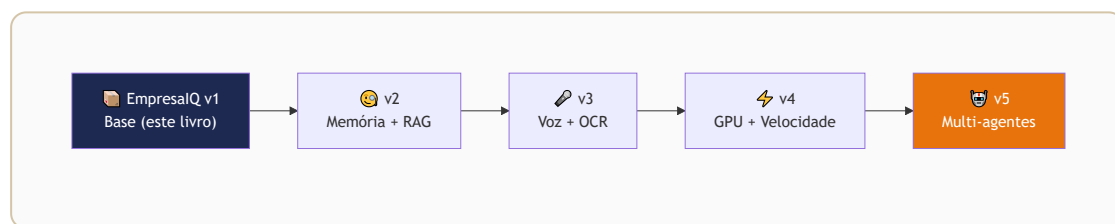
Lista de Verificação de Segurança

- ✓ Ambiente virtual isolado
- ✓ Servidor apenas em localhost
- ✓ Logs com rotação e limpeza automática
- ✓ Sem credenciais hardcoded no código
- ✓ Modelo GGUF verificado (hash SHA256)
- ✓ Actualizações regulares de dependências
- ✓ Acesso físico ao computador controlado

Melhorias Futuras

"O EmpresaIQ que construiu neste livro é a versão 1.0 — funcional e já útil. As próximas versões são o que o torna verdadeiramente poderoso."

O roteiro do EmpresaIQ



Cada versão acrescenta capacidades reais, sem substituir as anteriores. Pode implementar cada melhoria independentemente, conforme as necessidades da sua empresa.

Melhoria 1 — Memória vectorial com ChromaDB

A maior limitação do EmpresaIQ v1 é não se lembrar de documentos entre sessões. Com ChromaDB, o agente pode guardar e pesquisar documentos por **similaridade semântica** — mesmo sem ter visto exactamente as palavras usadas na pergunta.

```
pip install chromadb sentence-transformers
```

memoria_vetorial.py

```
import chromadb
from chromadb.utils import embedding_functions

# Base de dados vectorial local (persiste no disco)
cliente = chromadb.PersistentClient(path="./db_vetorial")

# Modelo de embeddings local (~80 MB, corre em CPU)
embeddings =
embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="all-MiniLM-L6-v2"
)

colecao = cliente.get_or_create_collection(
    name="documentos_empresaiq",
    embedding_function=embeddings
)

# Adicionar documento à memória
colecao.add(
    documents=["Contrato de prestação de serviços
EmpresaIQ..."],
    ids=["contrato_001"],
    metadatas=[{"tipo": "contrato", "data": "2026-01-15"}]
)

# Pesquisar por similaridade
resultados = colecao.query(
    query_texts=["qual é o prazo do contrato?"],
    n_results=3
)
print(resultados["documents"])
```

Capacidades desbloqueadas:

- Responder perguntas sobre documentos internos (RAG)
- Memória persistente entre sessões
- Pesquisa em bases de conhecimento com centenas de documentos

Melhoria 2 — Interface web completa com FastAPI

Evolua da interface Flask do capítulo 13 para uma API REST moderna:

```
pip install fastapi uvicorn
```

api_empresaiq.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from agente_local import agent_executor

app = FastAPI(title="EmpresaIQ Agent API", version="2.0")

class Pedido(BaseModel):
    pergunta: str

@app.post("/chat")
async def chat(pedido: Pedido):
    if not pedido.pergunta.strip():
        raise HTTPException(status_code=400,
detail="Pergunta vazia")
    resultado = agent_executor.invoke({"input":
pedido.pergunta})
    return {"resposta": resultado["output"]}

@app.get("/saude")
async def saude():
    return {"estado": "operacional", "modelo": "empresaiq"}

# Iniciar: uvicorn api_empresaiq:app --host 127.0.0.1 --port
8000
# Documentação automática: http://localhost:8000/docs
```

Melhoria 3 — OCR para analisar PDFs

Permita ao agente ler e analisar documentos PDF digitalizados:

```
pip install pytesseract pillow pdf2image
# Instalar também: Tesseract OCR (https://github.com/UB-Mannheim/tesseract/wiki)
```

```
from langchain.agents import tool
import pytesseract
from pdf2image import convert_from_path

@tool
def analisar_pdf(caminho_pdf: str) -> str:
    """Extraí e retorna o texto de um ficheiro PDF. Usa OCR se necessário."""
    imagens = convert_from_path(caminho_pdf)
    texto = ""
    for imagem in imagens:
        texto += pytesseract.image_to_string(imagem, lang='por')
    return texto[:3000] # Limita ao contexto disponível
```

Melhoria 4 — Reconhecimento de voz com Whisper

OpenAI Whisper é um modelo de transcrever voz que corre completamente offline:

```
pip install openai-whisper
```

voz.py

```
import whisper

modelo_whisper = whisper.load_model("small") # ~244 MB, bom equilíbrio qualidade/velocidade

def transcrever_audio(caminho_audio: str) -> str:
```

```
"""Transcreve um ficheiro de áudio para texto em
português."""
resultado = modelo_whisper.transcribe(caminho_audio,
language="pt")
return resultado["text"]

# Integrar no loop de chat do cap. 13:
# pergunta = transcrever_audio("gravacao.wav")
# resposta = agent_executor.invoke({"input": pergunta})
```

Melhoria 5 — Aceleração GPU (se tiver placa dedicada)

Se o servidor evoluir para ter uma GPU NVIDIA, a velocidade pode aumentar 10x-50x. O Ollama suporta CUDA nativamente — basta instalar o Ollama com suporte GPU:

```
# Linux com NVIDIA GPU (CUDA instalado)
# O Ollama detecta automaticamente a GPU ao iniciar
ollama serve

# Verificar se a GPU está a ser usada
ollama ps # Mostra o modelo activo e se usa GPU ou CPU
```

```
# No código Python, não precisa de mudar nada
# O Ollama gere automaticamente o offloading para GPU
from langchain_ollama import OllamaLLM

llm = OllamaLLM(
    model="empresaiq",
    base_url="http://localhost:11434",
    # Com GPU, as respostas são muito mais rápidas
    # automaticamente
)
```

Hardware	Velocidade aproximada
CPU i5 (baseline)	~3 tokens/seg
CPU i7 com AVX2	~6 tokens/seg
NVIDIA RTX 3060	~40 tokens/seg
NVIDIA RTX 4090	~200 tokens/seg

Melhoria 6 — Dashboard de monitorização com Streamlit

Visualize o histórico de uso do EmpresaIQ:

```
pip install streamlit pandas
```

dashboard.py

```
import streamlit as st
import pandas as pd
import json

st.title("🤖 EmpresaIQ – Painel de Monitorização")

# Carregar logs
tentativas = []
with open('log_agente.jsonl') as f:
    for linha in f:
        tentativas.append(json.loads(linha))

df = pd.DataFrame(tentativas)
df['timestamp'] = pd.to_datetime(df['timestamp'])

st.metric("Total de perguntas", len(df))
st.line_chart(df.set_index('timestamp').resample('H').size())
st.dataframe(df.tail(10))
```

```
# Executar: streamlit run dashboard.py
```

Resumo do roteiro

Versão	Melhoria	Esforço	Impacto
v1.1	Memória vectorial (ChromaDB)	Médio	Muito alto
v1.2	API REST com FastAPI	Fácil	Alto
v1.3	OCR para PDFs	Fácil	Alto
v2.0	Reconhecimento de voz	Médio	Alto
v2.1	GPU acceleration	Médio	Muito alto
v3.0	Dashboard Streamlit	Fácil	Médio

Capítulo seguinte: [17. Qwen2.5 no Agente EmpresaIQ](#) →

1. Memória Vetorial com ChromaDB

Permita que o agente "lembre" de documentos e pesquise por similaridade semântica:

```
pip install chromadb sentence-transformers
```

```
memoria_vetorial.py
```

```

import chromadb
from chromadb.utils import embedding_functions

# Criar base de dados vectorial local
cliente = chromadb.PersistentClient(path="./db_vetorial")

# Usar modelo de embeddings local (sem internet)
embeddings =
embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="all-MiniLM-L6-v2" # ~80 MB, corre em CPU
)

colecao = cliente.get_or_create_collection(
    name="documentos_empresaiq",
    embedding_function=embeddings
)

# Adicionar documento
colecao.add(
    documents=["Contrato de prestação de serviços
EmpresaIQ..."],
    ids=["contrato_001"],
    metadatas=[{"tipo": "contrato", "data": "2026-05-29"}]
)

# Pesquisar por similaridade
resultados = colecao.query(
    query_texts=["qual é o prazo do contrato?"],
    n_results=3
)

```

Capacidades desbloqueadas:

- Responder perguntas sobre documentos internos
- Pesquisa semântica em bases de conhecimento
- "Memória" persistente entre sessões

2. Interface Web Completa com FastAPI + React

```

pip install fastapi uvicorn

```


api.py

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="EmpresaIQ Agent API")

class Pergunta(BaseModel):
    texto: str

@app.post("/chat")
async def chat(pergunta: Pergunta):
    resposta = agent_executor.invoke({"input":
    pergunta.texto})
    return {"resposta": resposta["output"]}

# Iniciar: uvicorn api:app --host 127.0.0.1 --port 8000
```

3. OCR para Analisar Documentos PDF

```
pip install pytesseract pillow pdf2image
# Instalar Tesseract: https://github.com/UB-Mannheim/tesseract/wiki
```

```
@tool
def analisar_pdf(caminho_pdf: str) -> str:
    """Extrai e analisa o texto de um ficheiro PDF."""
    import pytesseract
    from pdf2image import convert_from_path

    imagens = convert_from_path(caminho_pdf)
    texto_completo = ""

    for imagem in imagens:
        texto_completo +=
        pytesseract.image_to_string(imagem, lang='por')

    return texto_completo[:3000] # Limita ao contexto
    disponível
```

4. Reconhecimento de Voz com Whisper

```
pip install openai-whisper
```

```
import whisper

modelo_whisper = whisper.load_model("small") # ~244 MB

def transcrever_audio(caminho_audio: str) -> str:
    resultado = modelo_whisper.transcribe(caminho_audio,
    language="pt")
    return resultado["text"]

# Integrar no loop de chat
import sounddevice as sd
import numpy as np
import scipy.io.wavfile as wav

def gravar_voz(segundos=5):
    """Grava áudio do microfone."""
    audio = sd.rec(int(segundos * 16000), samplerate=16000,
    channels=1)
    sd.wait()
    wav.write("temp_audio.wav", 16000, audio)
    return transcrever_audio("temp_audio.wav")
```

5. Síntese de Voz com Piper TTS

Piper é um sistema TTS local extremamente leve e com suporte a português:

```
pip install piper-tts
```

```
import subprocess

def falar(texto: str):
    """Converte texto em fala usando Piper TTS."""
    subprocess.run([
        "piper",
        "--model", "pt_PT-tugao-medium.onnx",
```

```
        "--output_file", "resposta.wav"
    ], input=texto.encode(), check=True)

    # Reproduzir o áudio
    subprocess.run(["aplay", "resposta.wav"]) # Linux
    # subprocess.run(["start", "resposta.wav"], shell=True)
    # Windows
```

6. Dashboard de Monitorização

Visualize métricas do agente com Streamlit:

```
pip install streamlit pandas
```

dashboard.py

```
import streamlit as st
import pandas as pd
import json

st.title("EmpresaIQ – Dashboard do Agente IA")

# Carregar logs
logs = []
try:
    with open("log_agente.jsonl") as f:
        for linha in f:
            logs.append(json.loads(linha))
except FileNotFoundError:
    st.warning("Sem logs registados ainda.")

if logs:
    df = pd.DataFrame(logs)
    st.metric("Total de perguntas", len(df))
    st.dataframe(df[["timestamp", "pergunta", "resposta"]])

# Iniciar: streamlit run dashboard.py
```

Roadmap Sugerido

Fase 1 (Semana 1-2):	✓ Agente base funcional
Fase 2 (Semana 3-4):	Memória vetorial com ChromaDB
Fase 3 (Mês 2):	Interface web FastAPI
Fase 4 (Mês 3):	OCR e análise de PDFs
Fase 5 (Mês 4):	Voz (Whisper + Piper)
Fase 6 (Mês 5-6):	Dashboard e monitorização

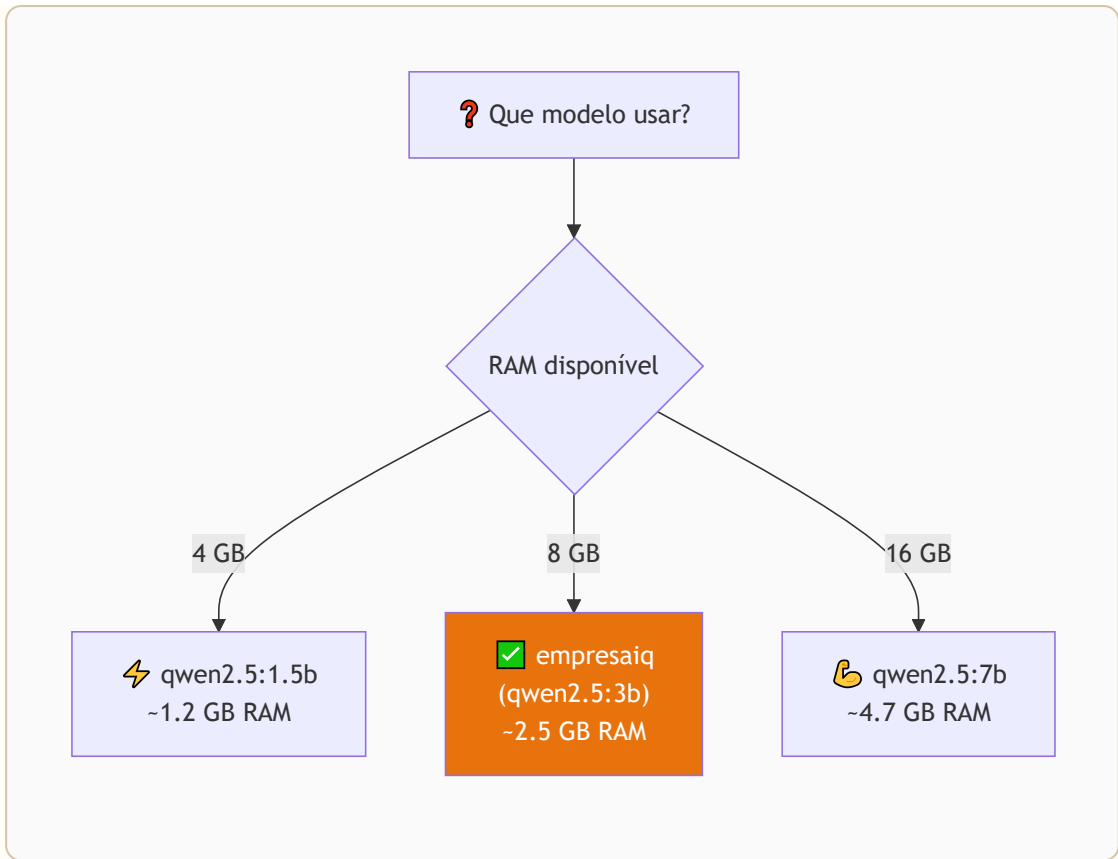
Modelos Alternativos via Ollama

"Modelos diferentes têm personalidades diferentes. Com o Ollama, mudar de modelo é tão simples como mudar uma linha no Modelfile — todo o resto permanece igual."

O que é o Qwen2.5?

O Qwen2.5 é uma família de modelos Open Source desenvolvida pela Alibaba Cloud. Nos benchmarks de seguimento de instruções e uso de ferramentas (tool calling), o Qwen2.5 destaca-se como uma das melhores opções para hardware limitado.

O EmpresaIQ usa o Qwen2.5-3B como base por defeito (Cap. 9). Mas o Ollama tem um catálogo completo de modelos — pode experimentar qualquer um com um único comando.



Modelos disponíveis no Ollama

Modelo	Comando pull	RAM usada	Pontos fortes
empresaiq (padrão)	(criado no Cap. 9)	~2.5 GB	Personalizado para PT empresarial
qwen2.5:1.5b	ollama pull qwen2.5:1.5b	~1.2 GB	Ultra-rápido, 4 GB RAM
qwen2.5:3b	ollama pull qwen2.5:3b	~2.5 GB	✅ Base do EmpresaIQ
qwen2.5:7b	ollama pull qwen2.5:7b	~4.7 GB	Mais capaz, 16 GB RAM
phi4-mini	ollama pull phi4-mini	~2.5 GB	Alternativa Microsoft
llama3.2:3b	ollama pull llama3.2:3b	~2.0 GB	Meta, rápido
mistral:7b	ollama pull mistral:7b	~4.1 GB	Europeu, forte em PT

Usar o Qwen2.5-7B para mais qualidade

Se tiver 16 GB de RAM e quiser respostas de maior qualidade:

```
# Descarregar o modelo maior  
ollama pull qwen2.5:7b
```

Criar um Modelfile para a versão 7B

Modelfile-7b

```
FROM qwen2.5:7b

SYSTEM """
És o EmpresaIQ Pro – um assistente de inteligência
artificial empresarial
especializado para empresas portuguesas.

Respondes sempre em português de Portugal, com alta precisão
e detalhe.

...
"""

PARAMETER temperature 0.1
PARAMETER top_p 0.9
PARAMETER num_ctx 8192
PARAMETER repeat_penalty 1.1
```

Criar o modelo:

```
ollama create empresaiq-pro -f Modelfile-7b
ollama run empresaiq-pro
```

Mudar o modelo no agente Python

A mudança no `agente_local.py` é uma única linha:

`agente_local.py` (apenas esta linha muda)

```
# ANTES – modelo base EmpresaIQ (Qwen2.5-3B)
llm = OllamaLLM(
    model="empresaiq",
    base_url="http://localhost:11434",
    temperature=0.1,
)

# DEPOIS – modelo Pro (Qwen2.5-7B)
```



```
llm = OllamaLLM(  
    model="empresaiq-pro",          # ← só esta linha muda  
    base_url="http://localhost:11434",  
    temperature=0.1,  
)
```

Todo o resto do código — ferramentas, prompt, executor — não muda.

Comparar modelos para o contexto empresarial PT

Característica	qwen2.5:3b	qwen2.5:7b	phi4-mini
Velocidade CPU	⚡⚡⚡ Bom	⚡⚡ Moderado	⚡⚡⚡ Bom
Seguimento de instruções	★★★★★	★★★★★	★★★★★
Uso de ferramentas (ReAct)	★★★★★	★★★★★	★★★★★
Multilíngue (PT/EN/ES)	★★★★★	★★★★★	★★★★★
Taxa de alucinação	Baixa	Muito baixa	Baixa
RAM necessária	~2.5 GB	~4.7 GB	~2.5 GB
Licença	Apache 2.0	Apache 2.0	MIT

Para a maioria dos casos de uso empresarial, o `qwen2.5:3b` (base do EmpresaIQ) é mais do que suficiente. O `qwen2.5:7b` justifica-se

quando a precisão é crítica e tem RAM disponível.

Experimento: testar rapidamente um novo modelo

Para testar um modelo sem criar um Modelfile completo:

```
# Testar directamente via Ollama
ollama run phi4-mini
>>> Olá! Responde em português. Qual a diferença entre IVA e IRC?
```

Ou via Python:

```
import ollama

# Testar modelo alternativo rapidamente
resposta = ollama.chat(
    model='phi4-mini',
    messages=[{
        'role': 'user',
        'content': 'Que tipo de tarefas empresariais podes
ajudar?'
    }]
)
print(resposta['message']['content'])
```

Gerir o espaço em disco

Os modelos ocupam espaço em disco. Para gerir:

```
# Ver todos os modelos e tamanhos
ollama list

# Remover um modelo que não usa
ollama rm qwen2.5:7b
```

Resumo

Neste capítulo:

- Explorámos o catálogo de modelos disponíveis no Ollama
- Criámos um Modelfile para o Qwen2.5-7B (versão Pro)
- Vimos que mudar de modelo é apenas uma linha de código
- Comparámos modelos para o contexto empresarial português

Com o Ollama, experimentar novos modelos é simples e reversível. O modelo base do EmpresaIQ (`qwen2.5:3b`) continua a ser a escolha certa para a maioria dos casos de uso.

No próximo capítulo, adicionamos memória conversacional ao EmpresaIQ — para o agente se lembrar do contexto entre perguntas.

Capítulo seguinte: *18. Memória Conversacional* →

Memória Conversacional

"A diferença entre um chatbot e um assistente inteligente é a memória. Um chatbot esquece cada frase que disse. Um assistente lembra-se de tudo o que importa."

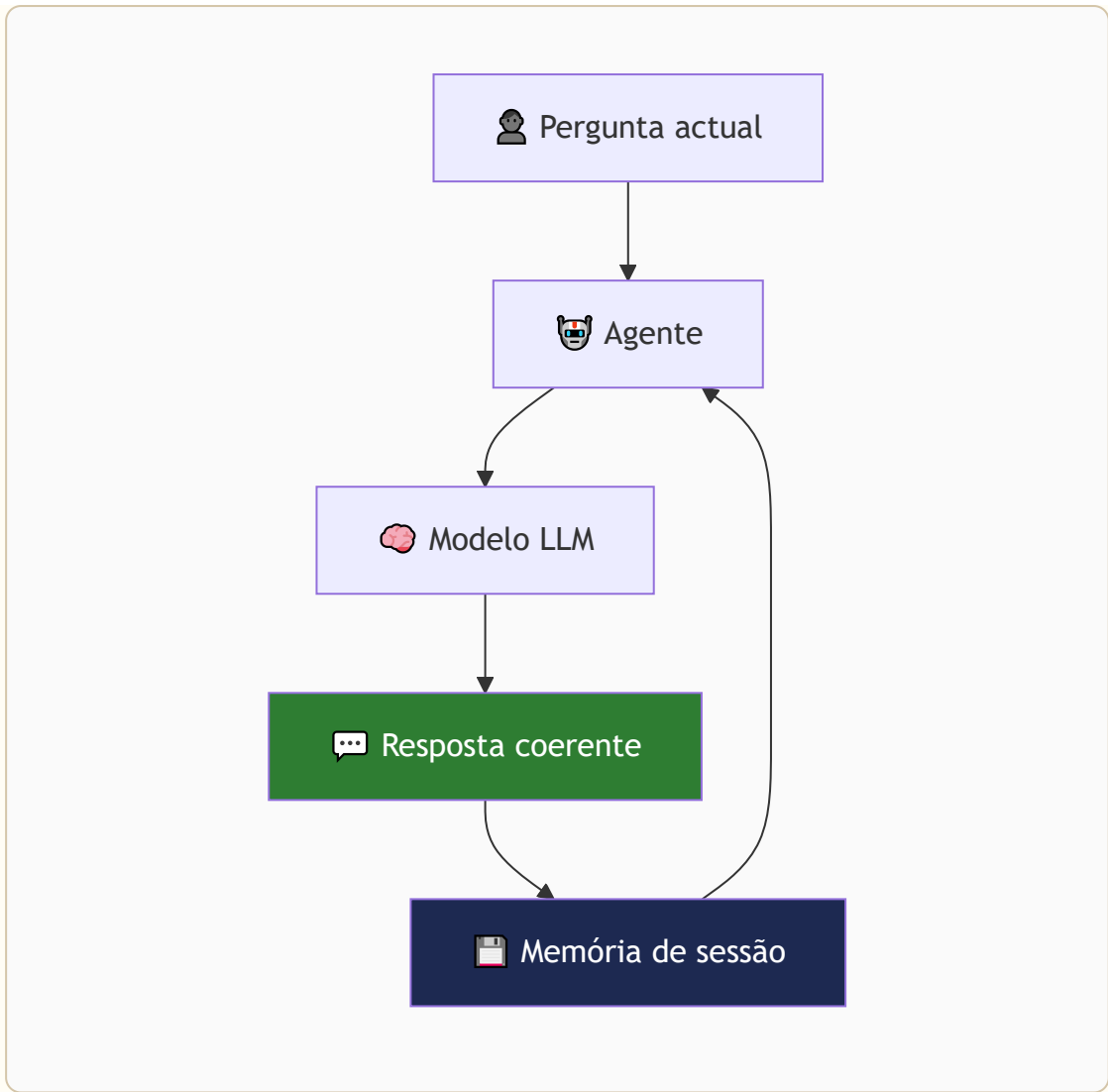
O problema: cada pergunta começa do zero

Por defeito, o EmpresaIQ trata cada pergunta de forma independente. Imagine esta conversa:

Você: "Tenho 10 contratos activos este mês."
Agente: "Percebido. Como posso ajudar?"

Você: "Quantos são de software?"
Agente: ??? ← Não sabe de que contratos está a falar

Sem memória, o agente não consegue referenciar mensagens anteriores. É como falar com alguém que perde a memória a cada frase.



Tipos de memória

Tipo	O que guarda	Persiste entre sessões?
Janela de contexto	Tokens actuais no prompt	Não (reset ao reiniciar)
Buffer Memory	Últimas N mensagens	Não (apenas na sessão)
Summary Memory	Resumo comprimido do histórico	Não (só na sessão)
Vector Memory (FAISS)	Embeddings de conversas passadas	Sim (persiste no disco)

Para o EmpresaIQ em hardware limitado, recomenda-se começar com **Buffer Window Memory** e evoluir para memória vectorial conforme a necessidade.

Opção 1 — Buffer Window Memory (recomendada para 8 GB RAM)

Guarda as últimas trocas (pergunta + resposta), descartando as mais antigas:

```
agente_com_memoria.py

from langchain.memory import ConversationBufferWindowMemory
from langchain_ollama import OllamaLLM
from langchain.agents import AgentExecutor,
create_react_agent
```

```

from langchain_core.prompts import PromptTemplate
from tools import consultar_portfolio_empresaiq,
calcular_iva

llm = OllamaLLM(
    model="empresaiq",
    base_url="http://localhost:11434",
    temperature=0.1
)

# Guarda as últimas 3 trocas (6 mensagens)
memory = ConversationBufferWindowMemory(
    k=3,
    memory_key="chat_history",
    return_messages=False # False = texto simples (menos
tokens)
)

tools = [consultar_portfolio_empresaiq, calcular_iva]

template = """
Sés o Agente EmpresaIQ. Respondes em português de Portugal.

Histórico recente:
{chat_history}

Ferramentas: {tools}
Nomes: {tool_names}

Pergunta: {input}
Thought: {agent_scratchpad}
"""

agent = create_react_agent(llm, tools,
PromptTemplate.from_template(template))

executor = AgentExecutor(
    agent=agent, tools=tools, memory=memory,
    verbose=False, max_iterations=3,
    handle_parsing_errors=True
)

# Testar a memória
perguntas = [
    "Tenho 10 contratos activos este mês.",
    "Quantos desses são de software?",
    "E qual é o preço do software da EmpresaIQ?"

```

```
]

for p in perguntas:
    print(f"\nVocê: {p}")
    r = executor.invoke({"input": p})
    print(f"Agente: {r['output']}")
```

⚠ Memória e consumo de contexto Cada mensagem no histórico ocupa tokens. Com `k=3` e mensagens médias, pode usar 400-800 tokens extra de contexto. Se o agente ficar mais lento, reduza `k=2`. ⚠

Opção 2 — Summary Memory (para conversas longas)

Em vez de guardar todas as mensagens na íntegra, o modelo resume automaticamente:

```
from langchain.memory import ConversationSummaryMemory

memory_resumo = ConversationSummaryMemory(
    llm=llm,          # Usa o mesmo modelo para resumir
    memory_key="chat_history"
)
```

Vantagem: Contém muito mais informação em menos tokens.

Desvantagem: Cada mensagem nova é resumida pelo modelo (mais lento).

Opção 3 — Memória vectorial com FAISS (persiste entre sessões)

Para o EmpresaIQ guardar memória mesmo depois de reiniciar o computador:

```
pip install faiss-cpu sentence-transformers
```

memoria_persistente.py

```
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import
HuggingFaceEmbeddings
from datetime import datetime
import os

# Modelo de embeddings local (~80 MB)
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

DB_PATH = "./memoria_empresaiq"

def guardar_na_memoria(pergunta: str, resposta: str):
    """Guarda uma troca na memória persistente."""
    texto = f"[{datetime.now().strftime('%Y-%m-%d')}]
Utilizador: {pergunta} | Agente: {resposta}"

    if os.path.exists(DB_PATH):
        db = FAISS.load_local(DB_PATH, embeddings,
allow_dangerous_deserialization=True)
        db.add_texts([texto])
    else:
        db = FAISS.from_texts([texto], embeddings)

    db.save_local(DB_PATH)

def pesquisar_na_memoria(pergunta: str, k: int = 3) -> str:
    """Recupera memórias relevantes para a pergunta
actual."""
    if not os.path.exists(DB_PATH):
        return ""
```

```
db = FAISS.load_local(DB_PATH, embeddings,
allow_dangerous_deserialization=True)
resultados = db.similarity_search(pergunta, k=k)
return "\n".join([r.page_content for r in resultados])
```

Arquitectura recomendada para o EmpresarialQ

Para a maioria das empresas, a combinação ideal é:

Sessão actual:
ConversationBufferWindowMemory (k=3)
→ Rápido, zero configuração extra

Entre sessões (opcional):
FAISS com embeddings locais
→ O agente "lembra" de conversações anteriores

Boas práticas

Prática	Porquê
Limite <code>k</code> a 3-5	Evita sobrecarga de RAM e contexto
Use <code>return_messages=False</code>	Menos tokens, mais eficiente
Separe memória por utilizador	Essencial para sistemas multi-utilizador
Não guarde dados sensíveis	Ou encripte o ficheiro FAISS

Resumo

Neste capítulo:

- Percebemos porque a memória transforma o EmpresaIQ num assistente verdadeiramente útil
- Implementamos `ConversationBufferWindowMemory` para contexto de sessão
- Explorámos FAISS para memória persistente entre sessões

No capítulo seguinte, vamos além da memória e adicionamos **estrutura de conhecimento** ao EmpresaIQ com ontologias.

*Capítulo seguinte: **19. Ontologias e Estrutura de Conhecimento** →*

Ontologias e Estrutura de Conhecimento

"Memória permite ao agente lembrar. Conhecimento permite ao agente compreender. A ontologia é o mapa que transforma dados soltos em inteligência estruturada."

O problema: dados sem estrutura

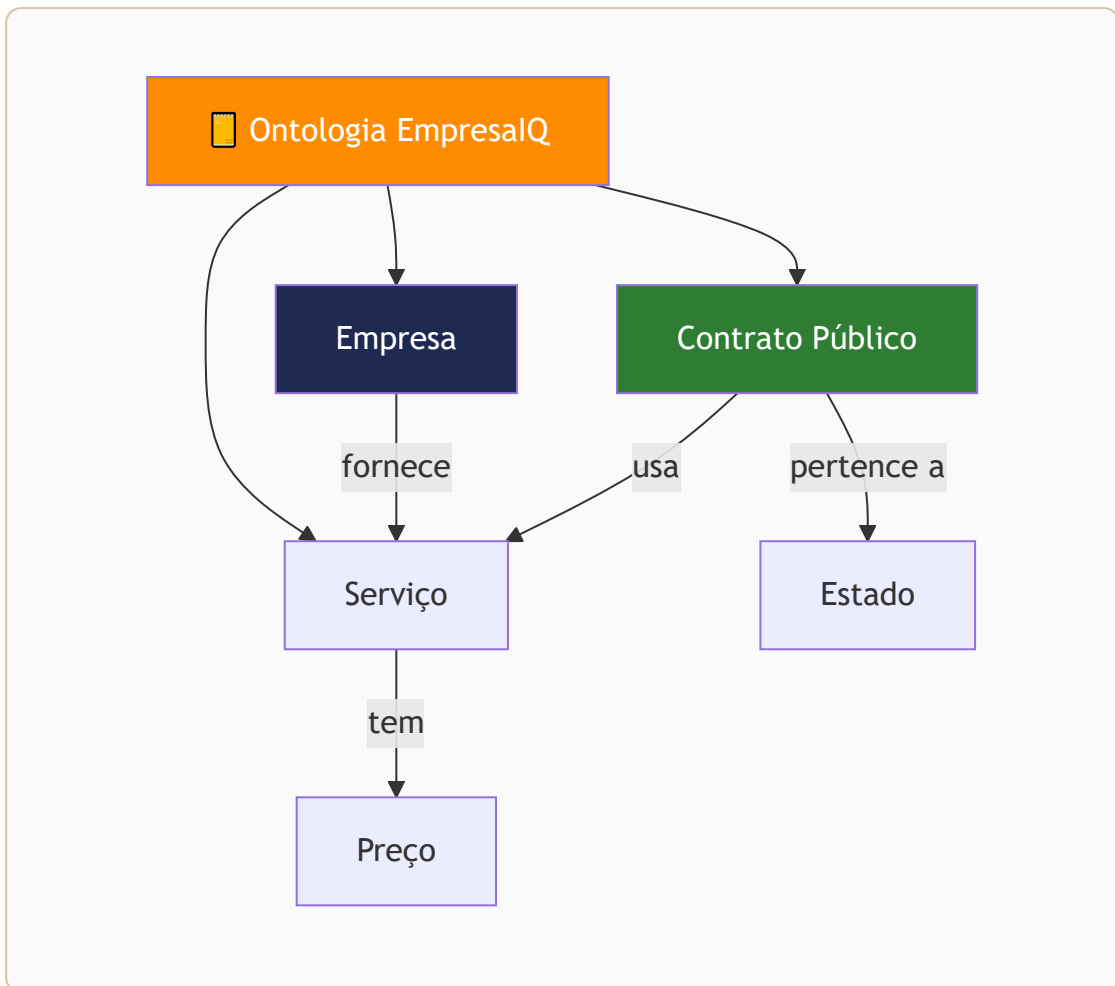
Sem ontologia, quando o EmpresaIQ lê este texto:

"A EmpresaIQ forneceu software ao contrato público B por 120.000€"

...o modelo interpreta-o como texto. Não "sabe" que:

- **EmpresaIQ** é uma empresa (entidade)
- **software** é um serviço (tipo)
- **contrato público B** é um objecto legal com regras próprias
- **120.000€** é um valor financeiro com implicações fiscais

Uma ontologia dá ao agente esse esqueleto lógico.



O que é uma ontologia

Uma ontologia é uma representação formal de conhecimento com três componentes:

Entidades (o que existe)

- Empresas, Contratos, Serviços, Clientes, Projectos, Documentos

Relações (como se ligam)

- Empresa **fornece** Serviço
- Cliente **contrata** Serviço
- Contrato **pertence a** Estado

- Projecto **inclui** Contrato

Regras (o que é permitido)

- Um contrato tem sempre uma entidade adjudicante
- Um serviço tem sempre um preço associado
- Uma empresa pode ter múltiplos contratos

Ontologia vs Memória vs RAG

Sistema	Função	Analogia
Memória	Lembrar interações	Diário de bordo
RAG	Recuperar documentos	Biblioteca de pesquisa
Ontologia	Estruturar conhecimento	Mapa conceitual

Os três sistemas são complementares. A ontologia é o esqueleto lógico que dá sentido ao que a memória lembra e o RAG recupera.

Implementação prática — Começar simples

Para o EmpresaIQ, a ontologia mais simples possível é um dicionário Python:

```
ontologia_empresaiq.py
```

```
# Ontologia EmpresaIQ v1 – estrutura mínima
```

```
ontologia = {
```

```

    "entidades": {
        "empresas": ["EmpresaIQ"],
        "servicos": ["software", "consultoria",
"cibersegurança", "formação"],
        "tipos_contrato": ["publico", "privado",
"framework"]
    },
    "relacoes": {
        "EmpresaIQ": {
            "fornece": ["software", "consultoria",
"cibersegurança", "formação"]
        }
    },
    "regras": {
        "contrato_publico": [
            "tem_entidade_adjudicante",
            "publicado_no_portal_base",
            "sujeito_a_iva_23"
        ]
    }
}

```

```

def classificar_entidade(texto: str) -> str:
    """Identifica o tipo de entidade num texto."""
    texto_lower = texto.lower()
    for servico in ontologia["entidades"]["servicos"]:
        if servico in texto_lower:
            return f"servico:{servico}"
    for empresa in ontologia["entidades"]["empresas"]:
        if empresa.lower() in texto_lower:
            return f"empresa:{empresa}"
    return "entidade:desconhecida"

# Exemplo de uso
print(classificar_entidade("preço da consultoria")) #
servico:consultoria
print(classificar_entidade("contrato EmpresaIQ")) #
empresa:EmpresaIQ

```

Integrar a ontologia no agente

A ontologia pode ser usada como contexto adicional no prompt do agente:

agente_com_ontologia.py (fragmento)

```
from ontologia_empresaiq import ontologia

# Gerar descrição concisa da ontologia para o prompt
def ontologia_para_prompt() -> str:
    servicos = ", ".join(ontologia["entidades"]["servicos"])
    return f"""
Estrutura de conhecimento EmpresaIQ:
- Serviços disponíveis: {servicos}
- Tipos de contrato: público, privado, framework
- Regra: contratos públicos estão sujeitos a IVA 23%
"""

# Incluir no template do agente
template = f"""
Sés o Agente EmpresaIQ.
{ontologia_para_prompt()}

Ferramentas: {{tools}}
Pergunta: {{input}}
Thought: {{agent_scratchpad}}
"""
```

Representação visual da ontologia EmpresaIQ

```
[EmpresaIQ]
├─ fornece → [Software]
├─ fornece → [Consultoria IA]
└─ oferece → [Cibersegurança]

[Contrato Público]
├─ pertence a → [Estado Português]
├─ usa → [Software]
└─ valor → [Preço + IVA 23%]
```



```
[Cliente]
  |— contrata → [Serviço]
  |— paga → [Factura]
```

Evolução futura: Graph Database

Para empresas com estruturas de dados mais complexas, a próxima evolução é usar uma base de dados de grafo como **Neo4j**:

```
# Exemplo conceptual – Neo4j com Python
from neo4j import GraphDatabase

driver = GraphDatabase.driver("bolt://localhost:7687", auth=
("neo4j", "password"))

with driver.session() as session:
    # Criar relação na ontologia
    session.run(
        "MERGE (e:Empresa {nome: $empresa}) "
        "MERGE (s:Servico {nome: $servico}) "
        "MERGE (e)-[:FORNECE]->(s)",
        empresa="EmpresaIQ", servico="Software"
    )
```

Benefícios reais da ontologia no EmpresalQ

Sem ontologia	Com ontologia
Respostas genéricas do LLM	Respostas baseadas em estrutura real
Confunde serviços com empresas	Identifica correctamente cada entidade
Inventa relações	Segue relações definidas explicitamente
Escalabilidade limitada	Cresce para sistemas complexos

Resumo

Neste capítulo:

- Percebemos o que é uma ontologia e porque transforma um chatbot num sistema de inteligência estruturada
- Criamos uma ontologia mínima para o EmpresalQ em Python
- Integrámos a ontologia no prompt do agente
- Explorámos a evolução para Neo4j em versões mais avançadas

Numa única frase: **Memória = experiência | RAG = conhecimento | Ontologia = estrutura.**

No último capítulo, fazemos um balanço completo do que construiu ao longo deste livro.

19.1 Introdução

Enquanto a memória permite ao agente "lembrar", a ontologia permite ao agente **compreender a estrutura do mundo**.

Uma ontologia define:

- O que existe no sistema (entidades)
- Como essas entidades se relacionam
- Que regras governam essas relações

No contexto do EmpresaIQ, isto transforma o agente de um simples LLM com ferramentas num verdadeiro **sistema de inteligência empresarial estruturada**.

19.2 O Problema sem Ontologia

Sem estrutura ontológica, um agente:

- Trata contratos como texto solto
- Mistura empresas, serviços e preços
- Não entende relações entre dados
- Depende apenas de embeddings ou prompts

Exemplo:

"Empresa A fornece software para contrato B"

Sem ontologia, o modelo não "sabe" que:

- Empresa A → entidade
 - Software → tipo de serviço
 - Contrato B → objeto legal
-

19.3 O que é uma Ontologia em IA

Uma ontologia é uma representação formal de conhecimento composta por:

◆ Entidades (Nodes)

- Empresas
- Contratos
- Serviços
- Pessoas
- Projetos

◆ Relações (Edges)

- fornece
- contrata
- pertence a
- compete com
- executa

◆ Regras

- Um contrato pertence a uma entidade pública

- Um serviço tem um preço associado
- Uma empresa pode ter múltiplos contratos

19.4 Ontologia vs RAG vs Memória

Sistema	Função
Memória	Lembrar interações
RAG	Recuperar informação textual
Ontologia	Estruturar conhecimento

👉 A ontologia é o "esqueleto lógico" do sistema.

19.5 Ontologia no EmpresarialQ

No EmpresarialQ, a ontologia pode ser definida como:

◆ Entidades principais

- Empresa
- Contrato Público
- Serviço
- Cliente
- Projeto
- Documento

◆ Relações principais

- Empresa → fornece → Serviço
- Cliente → contrata → Serviço
- Contrato → pertence a → Estado
- Projeto → inclui → Contrato

19.6 Representação simples (grafo)

```
[EmpresaIQ]
├─ fornece → [Software]
├─ fornece → [Consultoria IA]
└─ protege → [Sistemas Cibersegurança]

[Contrato Público]
├─ pertence a → [Estado]
├─ usa → [Software]
└─ valor → 120.000€
```

19.7 Implementação prática (Python)

Para o sistema EmpresaIQ, podemos começar com uma ontologia simples em dicionário:

```
ontology = {
    "Empresa": ["EmpresaIQ"],
    "Servicos": ["software", "consultoria",
"ciberseguranca"],
    "Contratos": [],
    "Relacoes": {
        "EmpresaIQ": ["fornece software", "fornece
consultoria", "fornece ciberseguranca"]
    }
}
```

19.8 Ontologia + Agente (Integração)

O agente pode usar ontologia para:

- Melhorar decisões de tools
- Validar respostas
- Reduzir erros de contexto
- Melhorar consistência

Exemplo de uso

Pergunta:

"O que faz a EmpresaIQ?"

Sem ontologia	Com ontologia
Resposta genérica do LLM	Resposta estruturada baseada em relações reais

19.9 Ontologia + RAG + Memória

Quando combinamos os 3 sistemas:

◆ Memória

- histórico do utilizador

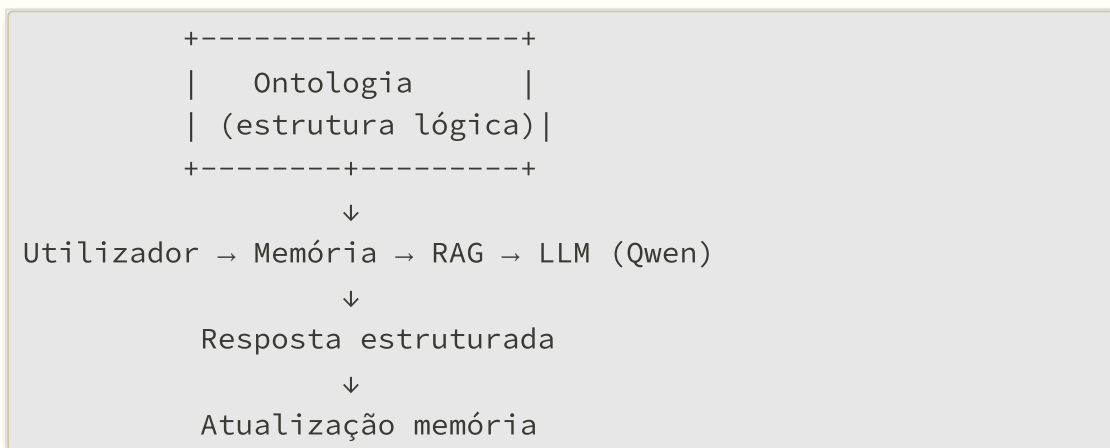
◆ RAG

- documentos reais

◆ Ontologia

- estrutura lógica do mundo

19.10 Arquitetura final EmpresarialQ



19.11 Benefícios reais

✓ Consistência lógica

O agente deixa de "inventar relações".

✓ Melhor tomada de decisão

Entende contexto empresarial real.

✓ Menos alucinação

As respostas seguem estrutura definida.

✓ Escalabilidade

Permite crescer para sistemas empresariais complexos.

19.12 Evolução futura da ontologia

Numa versão avançada do EmpresaIQ, a ontologia pode evoluir para:

- Graph Database (Neo4j)
 - Knowledge Graph semântico
 - Integração com RAG híbrido
 - Auto-expansão da estrutura
-

19.13 Conclusão

A ontologia representa o passo final para transformar o EmpresaIQ num sistema de inteligência estruturada.

Componente	Papel
Memória	experiência
RAG	conhecimento
Ontologia	estrutura

Então o agente torna-se um verdadeiro sistema cognitivo empresarial.

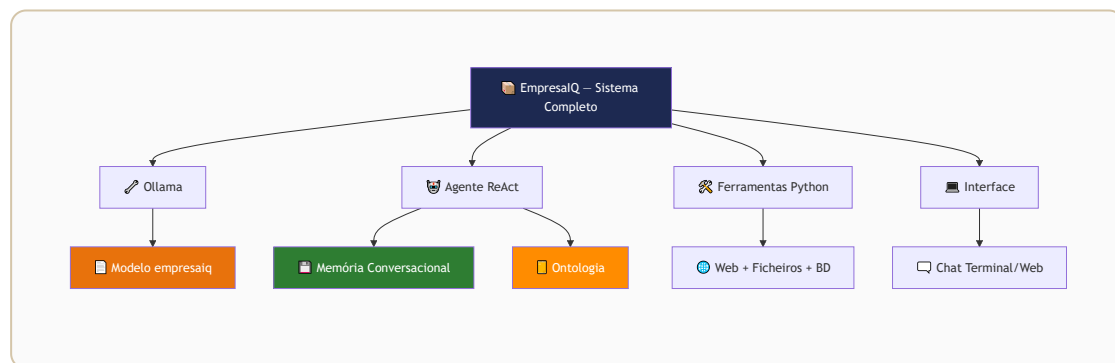
:::tip Com ontologia, o EmpresaIQ deixa de ser apenas um chatbot e passa a ser um **sistema de inteligência organizacional local completo**. :::

Conclusão

"O futuro da IA não pertence apenas à cloud. Pertence a quem tem controlo, privacidade e autonomia."

O que construiu

Ao longo deste livro, construiu um agente de inteligência artificial completo que corre **100% localmente** num PC normal com apenas 8 GB de RAM. Sem subscrições, sem enviar dados para servidores externos, sem dependência de internet.



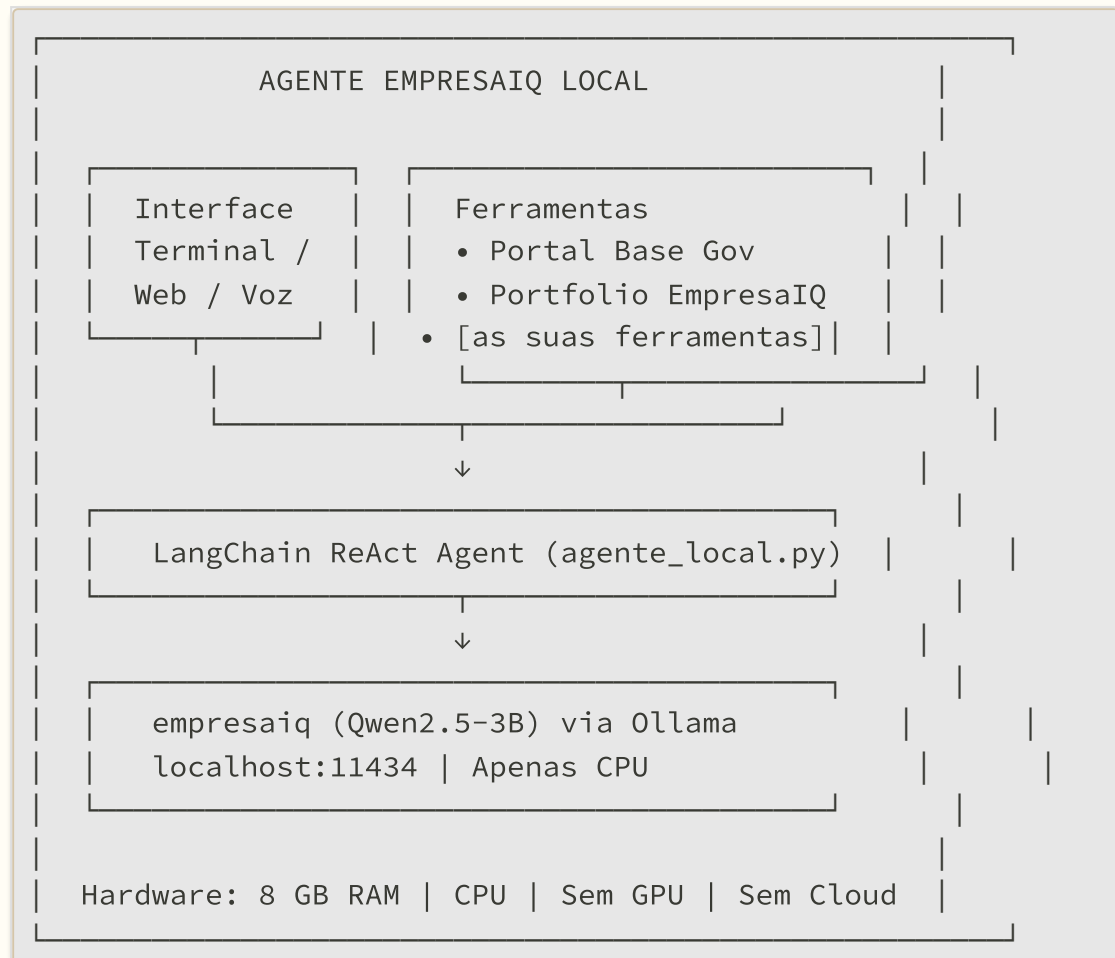
O que aprendeu — mapa completo

Parte	Capítulos	Conhecimento adquirido
Fundamentos	1–2	Porquê IA local é estratégica para empresas portuguesas
Hardware	3–4	Como escolher modelos para hardware limitado, sem GPU
Técnica	5	GGUF e quantização — o que acontece dentro do ficheiro
Setup	6–9	Instalação completa: Python, Ollama, modelo EmpresaIQ
Construção	10–12	Ferramentas, agente ReAct, optimizações CPU
Produção	13–16	Interface, automatização, segurança, melhorias
Avançado	17–19	Qwen2.5, memória, ontologias

Os quatro pilares do EmpresaIQ

1. MODELO CERTO	→	empresaiq (Qwen2.5-3B)	
2. FORMATO CERTO	→	Ollama Modelfile	
3. MOTOR CERTO	→	Ollama (servidor local de IA)	
4. OPTIMIZAÇÃO	→	OLLAMA_KEEP_ALIVE + num_ctx	

Arquitetura final do EmpresaIQ



O que pode fazer a seguir

Próximos passos concretos para o EmpresaIQ:

- **Adicionar ferramentas** específicas ao seu negócio (ERP, CRM, base de dados interna)
- **Integrar documentos** com ChromaDB ou FAISS para RAG
- **Criar uma interface web** profissional com autenticação
- **Automatizar tarefas** repetitiáveis com cron / Task Scheduler
- **Expandir para voz** com Whisper (entrada) + Piper TTS (saída)
- **Evoluir a ontologia** para um Knowledge Graph com Neo4j

- **Combinar Memória + RAG + Ontologia** num sistema cognitivo empresarial completo
-

Mensagem final

Quando começou este livro, talvez IA local parecesse algo para grandes empresas com servidores dedicados. Agora sabe que não é verdade.

Com um PC de escritório, Python, e os modelos e ferramentas certos, qualquer empresa — do advogado independente ao retalhista local — pode ter um agente inteligente a funcionar **hoje, sem custos de subscrição, sem expor dados a terceiros.**

:::tip O EmpresaIQ que construiu é seu O código, os dados, o modelo — estão todos no seu computador. Não há termos de serviço que mudam, não há preços que sobem, não há API que deixa de funcionar. A inteligência é sua. :::

Recursos úteis

Recurso	Link
Ollama	ollama.com
LangChain	python.langchain.com
Qwen2.5 (Alibaba)	huggingface.co/Qwen
LangChain Ollama	github.com/langchain-ai/langchain
ChromaDB	trychroma.com
Whisper	github.com/openai/whisper
Piper TTS	github.com/rhasspy/piper

Guia produzido pela **EmpresaIQ** — Inteligência Empresarial & IA
Versão 2.0 — 2026

CONTRACAPA

O controlo da IA fica consigo

O código, os dados e o modelo ficam no seu computador. A próxima evolução do EmpresarialQ pode crescer consigo, sem depender da cloud.

© 2026 EmpresarialQ